Implementation of Ransomware Protection Using Snapshots, WORM Files and Fpolicy

A PROJECT REPORT

Submitted by

Joilin S M	**111722202018**
Dharsan S	**111722202007**
Kevin Isaac E	**111722201010**

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND BUSINESS SYSTEMS

R.M.K. ENGINEERING COLLEGE

(An Autonomous Institution)

R.S.M. Nagar, Kavaraipettai-601 206



March, 2026

R.M.K. ENGINEERING COLLEGE

(An Autonomous Institution)

R.S.M. Nagar, Kavaraipettai-601 206

BONAFIDE CERTIFICATE

Certified that this project report **"Implementation of Ransomware Protection Using Snapshots, WORM Files and Fpolicy"** is the Bonafide work of **Joilin S M (111722202018), Dharsan S (111722202007) and Kevin Isaac E (111722202010),** who carried out the **22CB811 Project Work** under my supervision.

SIGNATURE

Dr. K. Chidambarathanu,
Head of the Department
Department of Computer Science and Business Systems,
R.M.K. Engineering College,
R.S.M. Nagar, Kavaraipettai,
Tiruvallur District– 601 206.

SIGNATURE

Dr. V. R. Kavitha
Associate Professor
Department of Computer Science and Business Systems,
R.M.K. Engineering College,
R.S.M. Nagar, Kavaraipettai,
Tiruvallur District– 601 206.

Submitted for the Project Viva –Voce held……. at **R.M.K. Engineering College**, Kavaraipettai, Tiruvallur District– 601206.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We earnestly portray our sincere gratitude and regard to our beloved **Chairman Shri. R. S. Munirathinam, our Vice Chairman, Shri. R. M. Kishore** and **our Director, Shri. R. Jyothi Naidu,** for the interest and affection shown towards us throughout the course.

We convey our sincere thanks to our **Principal**, **Dr. K. A. Mohamed Junaid,** for being thesource of inspiration in this college.

We reveal our sincere thanks to our **Professor and Head of the Department, Computer Science and Business Systems, Dr. K. Chidambarathanu,** for his commendable support and encouragement for the completion of our project.

We would like to express our sincere gratitude for our Project Coordinator and Project Guide **Dr. V.R. Kavitha, Associate Professor** for her valuable suggestions towards the successful completion for this project in a global manner.

We take this opportunity to extend our thanks to my Parents, all faculty members of Department of Computer Science and Business Systems and friends for all that they mean to us during the crucial times of the completion of our project.

ABSTRACT

Ransomware has emerged as a critical cybersecurity threat, causing severe data loss, operational disruption, and financial damage to organizations by encrypting or deleting sensitive files stored in enterprise systems. Traditional security mechanisms often fail to provide comprehensive protection against rapidly evolving ransomware techniques, particularly those targeting file-based storage environments. To address this challenge, a multi-layered ransomware protection framework is designed and implemented using enterprise storage technologies such as FPolicy, SnapLock (WORM), and snapshot mechanisms within a NetApp ONTAP environment. The solution leverages real-time behavioral monitoring through FPolicy to detect and block suspicious file activities, enforces data immutability using SnapLock WORM to prevent unauthorized modification or deletion, and utilizes periodic snapshots for rapid point-in-time recovery in the event of encryption. By integrating proactive detection, preventive controls, and efficient recovery strategies, the framework establishes a defense-in-depth approach that enhances data integrity, minimizes downtime, and strengthens overall organizational resilience against ransomware attacks.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	ABSTRACT	iii
	LIST OF FIGURES	vii
	LIST OF TABLES	ix
	LIST OF SYMBOLS	xiii
1.	**INTRODUCTION**	
	1.1 Problem Statement	1
	1.2 Literature Survey	1
	1.3 System Requirements	4
	1.3.1 Hardware Requirements	4
	1.3.2 Software Requirements	5
	1.3.3 Feasibility Study	6
2.	**SYSTEM ANALYSIS**	8
	2.1 Existing System	8
	2.1.1 Disadvantages of Existing System	8
	2.2 Proposed System	9
	2.2.1 Advantages of Proposed System	10

3. SYSTEM DESIGN 11

3.1 System Architecture 11

3.2 UML Diagrams 13

3.2.1 Use Case Diagram 13

3.2.2 Class Diagram 15

3.2.3 Activity Diagram 17

3.2.4 Sequence Diagram 18

4. SYSTEM IMPLEMENTATION 20

4.1 Modules 20

4.2 Module description 20

4.2.1 System Hardening 20

4.2.2 Configuration Monitoring 24

4.2.3 Data Protection 28

4.3 Algorithms 33

5. TESTING 34

5.1 Unit Testing 3

5.1.1 Test Cases 34

5.2 Integration Testing 36

5.3 System Testing 36

6.	**RESULTS & DISCUSSION**	**37**
7.	**CONCLUSION & FUTURE ENHANCEMENTS**	**39**
	REFERENCES	**43**
	APPENDIX I – SOURCE CODE	**45**
	APPENDIX II – SCREENSHOTS	**50**

LIST OF FIGURES

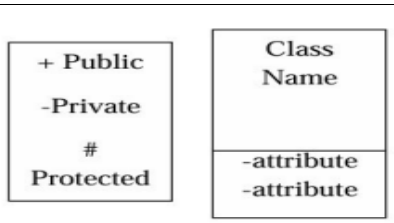
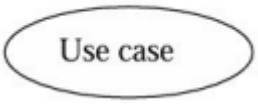
S.NO.	FIGURE NO.	DESCRIPTION	PAGE NO.
1.	3.1	Architecture Diagram of Ransomware Protection Using Snapshots, Worm Files And Fpolicy	11
2.	3.2.1	Use Case Diagram of Ransomware Protection Using Snapshots, Worm Files And Fpolicy	13
3.	3.2.2	Class Diagram of Ransomware Protection Using Snapshots, Worm Files And Fpolicy	15
4.	3.2.3	Activity Diagram of Ransomware Protection Using Snapshots, Worm Files And Fpolicy	17
5.	3.2.4	Sequence Diagram of Ransomware Protection Using Snapshots, Worm Files And Fpolicy	19
6.	4.2.1.1	Use Case Diagram of File Monitoring and Detection	21
7.	4.2.1.2	Class Diagram of File Monitoring and Detection	22
8.	4.2.1.3	Activity Diagram of File Monitoring and Detection	23
9.	4.2.1.4	Sequence Diagram of File Monitoring and Detection	24
10.	4.2.2.1	Use Case Diagram of Behavioral Analysis and Anomaly Detection	25
11.	4.2.2.2	Class Diagram of Behavioral Analysis and Anomaly Detection	26
12.	4.2.2.3	Activity Diagram of Behavioral Analysis and Anomaly Detection	27
13.	4.2.3.1	Sequence Diagram of Behavioral Analysis and Anomaly Detection	28
14.	4.2.3.1	Use Case Diagram of Data Immutability	28
15.	4.2.3.2	Class Diagram of Data Immutability	29
16.	4.2.3.3	Activity Diagram of Data Immutability	30
17.	4.2.3.4	Sequence Diagram of Data Immutability	31
18.	4.2.3.1	Use Case Diagram of Snapshot & Recovery	28
19.	4.2.3.2	Class Diagram of Data Snapshot & Recovery	29

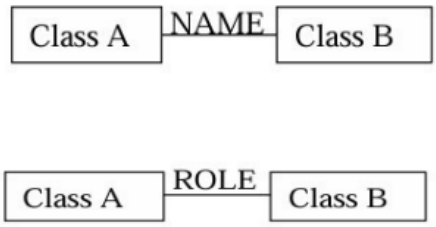
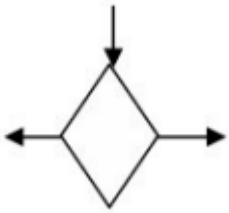
20.	4.2.3.3	Activity Diagram of Snapshot & Recovery	30
21.	4.2.3.4	Sequence Diagram of Snapshot & Recovery	31

LIST OF TABLES

S.NO.	TABLE NO.	DESCRIPTION	PAGE NO.
1	1.1	Hardware Requirements	12
2	1.2	Software Requirements	13
3	5.1	Test Cases	24

LIST OF SYMBOLS

S.NO	NOTATION NAME	NOTATION	DESCRIPTION
1.	Class		Represents a collection of similar entities grouped together.
2.	Actor		Specifies a role played by a user that interacts with the subject.
3.	Communication		Communication between various use cases.
4.	Transition		Represents communication that occurs between Processes.
5.	Use case		Interaction between the system and external environment.
6.	External entity		Represents external entities such as keyboard, sensors, etc
7.	Control Flow		Represents various control flow between the states.

8.	Association		Associations represent static relationships between classes Roles represent the way the two classes see each other.
9.	Initial State		Initial state of the object.
10.	Final state		Final state of the object.
11.	Decision box		Represents decision making process from a constraint.
12.	Object Lifeline		Represents the vertical dimensions that the object communications.
13.	Message		Represents the message exchanged.

CHAPTER 1

INTRODUCTION

1.1 Problem Statement

Modern organizations increasingly rely on network-attached storage systems to manage critical and sensitive data, making them prime targets for sophisticated ransomware attacks that rapidly encrypt, rename, or delete files, leading to severe data loss, operational disruption, and financial damage. Traditional signature-based security solutions are often ineffective against zero-day and evolving ransomware variants, as they fail to detect abnormal behavioral patterns in real time. Additionally, the absence of strong data immutability and efficient recovery mechanisms further increases the risk of permanent data compromise. Therefore, there is a critical need for an intelligent, multi-layered ransomware protection system that combines real-time file monitoring, behavioral anomaly detection, data immutability enforcement (WORM), and automated snapshot-based recovery to proactively detect threats, prevent unauthorized modifications, and ensure rapid restoration of affected data to maintain business continuity.

1.2 Literature survey

[1] Furnell, S., Emm, D. "The ABC of Ransomware Protection", *Computer Fraud & Security*, 2017, vol. 2017, no. 10, pp. 5–11.

This paper discusses the rapid growth of ransomware as a major cybersecurity threat causing data loss and operational disruption. It highlights three essential defensive measures: Anti-malware, Backup, and Critical patching (ABC). These mechanisms aim to reduce vulnerabilities and improve organizational preparedness. The study emphasizes proactive protection rather than reactive incident response. It explains how preventive controls can limit

ransomware infection and spread. The authors stress the importance of structured security frameworks in enterprise environments. These approaches support monitoring and protection of critical storage systems. The work provides foundational concepts for building layered ransomware defense strategies.

[2] Microsoft. "Microsoft Security Bulletin MS17-010 – Critical Security Update for Microsoft Windows SMB Server", *Microsoft Security Bulletin*, 2017, vol. 17, no. 1, pp. 1–8.

This bulletin explains the vulnerability in Windows SMB exploited by WannaCry ransomware. It highlights how unpatched systems become entry points for large-scale ransomware attacks. The document emphasizes the importance of timely security updates and vulnerability management. It shows how patch management is essential for preventing exploitation. The analysis demonstrates the risks posed to enterprise systems storing sensitive data. Continuous monitoring and security updates are recommended to reduce attack surfaces. The report supports the implementation of proactive defensive controls. These measures strengthen ransomware protection in enterprise storage infrastructures.

[3] Loscocco, P., Smalley, S. "Security-Enhanced Linux: Lessons in Mandatory Access Control", *IEEE Transactions on Secure Computing*, 2005, vol. 2, no. 1, pp. 34–45.

This paper introduces Security-Enhanced Linux (SELinux) and its implementation of Mandatory Access Control. It focuses on enforcing strict security policies within operating systems. The approach limits unauthorized privilege escalation and restricts system access. Such controls reduce vulnerabilities that attackers can exploit. Policy-based access enforcement improves protection of critical data environments. The study highlights how strong access control enhances system security. These concepts are relevant for protecting enterprise storage systems from ransomware. The work supports secure system design through strict policy enforcement.

[4] Ivanov, A., Emm, D., Sinitsyn, F., Pontiroli, S. "Kaspersky Security Bulletin 2016: The Ransomware Revolution", *Securelist Security Report*, 2016, vol. 8, no. 2, pp. 1–15.

This report analyzes the rapid evolution of ransomware families across global cyber threats. It explains the economic model behind crypto-ransomware attacks targeting organizational data. The study highlights how encryption-based attacks cause operational disruption and financial damage. It examines trends in ransomware distribution and infection strategies. The report stresses the importance of early detection and preventive security mechanisms. Organizations must implement strong monitoring to identify suspicious activity quickly. Proactive security measures help minimize data loss during ransomware incidents. The findings emphasize the need for multi-layer protection strategies.

[5] National Cyber Security Centre. "Ransomware: Latest Guidance", ***NCSC Security Report*****, 2017, vol. 5, no. 1, pp. 10–18.**

This guidance outlines recommended strategies for preventing ransomware attacks in organizations. It emphasizes the importance of reliable backups and secure data storage. Patch management is highlighted as a key defense against known vulnerabilities. Network segmentation and least-privilege principles improve system security. The guidance promotes preventive security rather than reactive recovery. Monitoring mechanisms are recommended to detect suspicious file activity. These strategies support real-time protection of enterprise storage environments. The recommendations align with structured ransomware defense frameworks.

[6] Zekrifa, D. M. S., Ghorbani, A. "Ameliorate Competitive Learning Neural Networks for System Intrusion Detection", ***Neurocomputing*****, 2013, vol. 140, no. 1, pp. 135–145.**

This paper presents improvements in neural network-based intrusion detection systems. It focuses on identifying abnormal system behavior caused by malicious activity. The proposed approach enhances detection accuracy and classification efficiency. Behavioral analysis helps identify unknown or emerging cyber threats. Such detection techniques are useful for recognizing ransomware activity patterns. Early identification of suspicious operations can prevent large-scale data encryption. Intelligent monitoring improves overall system security and resilience. The research supports advanced threat detection mechanisms for enterprise environments.

[7] Sinitsyn, F. "Locky: The Encryptor Taking the World by Storm", *Securelist Threat Research*, 2016, vol. 4, no. 2, pp. 22–30.

This study analyzes the Locky ransomware family and its attack mechanisms. It examines infection vectors such as phishing emails and malicious downloads. The research highlights advanced encryption techniques used by ransomware. It explains how rapidly these threats spread across enterprise systems. The analysis shows the importance of detecting abnormal file activities early. Real-time monitoring at the file system level is critical for prevention. Security mechanisms must respond quickly to stop mass encryption. The study emphasizes the need for proactive ransomware defense strategies.

[8] Goodin, D. "WannaCry Ransomware Spread Widely Because of Windows 7, Not XP", *Ars Technica Security Review*, 2017, vol. 9, no. 3, pp. 40–46.

This article investigates the widespread impact of the WannaCry ransomware outbreak. It reveals that many affected systems were patchable versions of Windows. Poor patch management and administrative negligence contributed to the attack. The study highlights the consequences of ignoring security updates. Organizations must implement continuous vulnerability management practices. Proactive monitoring helps detect exploitation attempts early. Strong security governance improves protection of enterprise systems. The findings reinforce the importance of preventive cybersecurity strategies.

[9] Securelist. "Schroedinger's Pet(ya)", *Securelist Threat Intelligence Report*, 2017, vol. 6, no. 2, pp. 12–20.

This paper analyzes the NotPetya cyberattack and its destructive behavior. It explains how the malware acted as a wiper disguised as ransomware. The attack used advanced exploitation techniques to spread within networks. Lateral movement enabled rapid compromise of enterprise systems. The incident caused major operational disruption and data loss. The study highlights the growing sophistication of ransomware-like threats. Organizations must adopt real-time monitoring and preventive security mechanisms. Advanced storage protection strategies are essential for minimizing ransomware impact.

1.3 System Requirements

1.3.1 Hardware Requirements

Table 1.1 Hardware requirements

1	**Processor (CPU)**	Minimum dual-core processor (Intel i3 / AMD Ryzen 3 or above)
2	**GPU**	Dedicated GPU with CUDA support and at least 4 GB VRAM
3	**RAM**	Minimum 4 GB RAM (8 GB recommended for multitasking and testing)
4	**Storage**	At least 40 GB free disk space (SSD preferred for faster execution)

` **1.2.1 Software Requirements**

Table 1.2 Software requirements

1	**Operating System**	Linux-based distribution (Ubuntu, Debian, Fedo deployment and testing
2	**Shell Environment**	Bash shell (v4.0 or later) for automation scripting
3	**Programming Utilities**	Core GNU/Linux utilities like `awk`, `sed`, `grep`, `find`, `cron`, `systemctl`, `ufw`, and `iptables`
4	**Storage OS**	NetApp ONTAP operating system (version supporting FPolicy services)
5	**Security and Monitoring Tools**	Tools like `auditd`, `chkrootkit`, `fail2ban`, and `aide` for system auditing and protection
6	**Version Control System**	Git for managing and tracking project versions during development

1.3.3 Feasibility Study

1. Technical Feasibility:

The proposed ransomware protection system using FPolicy is technically feasible as it leverages existing storage-level monitoring capabilities provided by NetApp ONTAP along with standard Linux-based administration tools. FPolicy supports real-time file event monitoring, policy enforcement, and integration with external servers, making it suitable for detecting abnormal file access patterns associated with ransomware. The required hardware and software components, including Linux operating systems, Bash scripting, logging utilities, and network configuration tools, are widely available and do not demand specialized infrastructure. Since the implementation primarily involves configuration, policy definition, event monitoring, and automation scripting rather than developing a system from scratch, the project can be executed efficiently within a controlled enterprise or lab environment.

2. Economic Feasibility:

The proposed ransomware protection system using FPolicy is economically feasible as it primarily utilizes existing infrastructure components such as NetApp ONTAP storage systems and Linux-based servers, reducing the need for significant additional investment. The implementation focuses on configuration, monitoring, and automation rather than developing costly proprietary software. Since most tools involved, including Linux utilities and scripting environments, are open-source and freely available, operational costs remain minimal. Moreover, by preventing ransomware attacks and minimizing downtime, data loss, and recovery expenses, the system can significantly reduce potential financial losses for organizations, making it a cost-effective security enhancement solution.

3. Operational Feasibility:

The proposed ransomware protection system is operationally feasible as it combines FPolicy monitoring, snapshot technology, and WORM (Write Once Read Many) file protection within the existing storage infrastructure. FPolicy enables real-time detection of suspicious file activities, while snapshots provide quick data recovery by maintaining point-in-time copies of files. WORM protection ensures that critical data cannot be modified or deleted for a defined retention period, preventing ransomware from encrypting or tampering with protected files. These mechanisms operate at the storage level without disrupting normal user activities, and they can be managed through standard administrative interfaces. Since the solution integrates with routine IT operations such as backup management, policy enforcement, and monitoring, it can be efficiently deployed and maintained within an organization's existing operational framework.

4. Legal and Regulatory Feasibility:

The proposed ransomware protection system is legally and regulatorily feasible as it supports key data protection and cybersecurity compliance requirements. By integrating FPolicy monitoring, snapshot-based recovery, and WORM file protection, the system ensures data integrity, availability, and secure retention. These mechanisms align with standards such as ISO/IEC 27001 and general data protection regulations that emphasize access control, logging, backup, and tamper-proof storage. Additionally, audit logs and immutable records assist in legal investigations and compliance reporting, making the solution suitable within existing regulatory frameworks.

5. Schedule Feasibility:

The project is feasible within the given time frame as it uses well-known technologies and structured development phases. The work is divided into requirement analysis, system design, implementation, testing, and documentation. Each phase is planned with realistic deadlines, ensuring smooth progress without delays. Since the tools and technologies are familiar, the development and testing can be completed efficiently within the scheduled timeline.

CHAPTER 2

SYSTEM ANALYSIS

2.1 Existing System

The existing system for ransomware protection primarily relies on traditional antivirus software and signature-based detection mechanisms to identify known malware threats. These systems scan files periodically and compare them against predefined malware signatures, making them effective only against previously identified ransomware variants. In most cases, detection occurs after files have already been encrypted or compromised, leading to significant data loss and downtime. Additionally, many conventional systems lack real-time behavioral monitoring, advanced anomaly detection, data immutability mechanisms, and automated snapshot-based recovery features. As a result, they are limited in their ability to prevent zero-day attacks, rapidly evolving ransomware strains, and large-scale file encryption within network-attached storage environments.

2.2 Disadvantages of Existing System

Existing ransomware protection systems have several limitations in handling modern threats. Most rely on signature-based detection, making them ineffective against new or zero-day ransomware variants. They often detect attacks only after encryption has occurred, leading to data loss and downtime. Additionally, many lack real-time behavioral monitoring, data immutability, and secure recovery mechanisms, which reduces their ability to prevent large-scale damage and ensure quick restoration.

2.3 Proposed System

The proposed system implements a multi-layered ransomware protection framework that integrates real-time file monitoring, behavioral anomaly detection, data immutability through WORM (SnapLock) protection, and automated snapshot-based recovery. It continuously monitors file operations to identify suspicious activities such as mass renaming, rapid encryption, or abnormal access patterns, enabling early threat detection. By enforcing retention policies and preventing unauthorized file modification or deletion, the system ensures data integrity even during active attacks. Additionally, periodic snapshots allow quick restoration of clean data, minimizing downtime and data loss. This proactive and defense-in-depth approach significantly enhances overall storage security and business continuity.

2.4 Advantage of Proposed System

The proposed system offers several advantages by providing proactive and multi-layered protection against ransomware attacks. It enables real-time monitoring and behavioral analysis to detect suspicious activities early, reducing the risk of large-scale data encryption. The implementation of WORM-based data immutability prevents unauthorized modification or deletion of critical files, ensuring data integrity even during active attacks. Automated snapshot creation and rapid recovery mechanisms minimize downtime and data loss, supporting business continuity. Overall, the system enhances reliability, resilience, and security compared to traditional reactive protection methods.

CHAPTER 3
SYSTEM DESIGN

3.1 System Architecture

1. User Interface Layer (UI):

• Provides a graphical or command-line interface for administrators to configure and manage ransomware protection policies.

• Allows administrators to configure FPolicy rules, define WORM retention periods, and schedule snapshot policies.

• Enables manual snapshot restoration and generation of security and compliance reports.

2. Application / Protection Layer:

• **File Monitoring & Detection Module:**

Continuously monitors file operations such as creation, deletion, renaming, and modification using FPolicy. Detects suspicious activities like rapid encryption, mass file changes, and abnormal write patterns.

• **Behavioral Analysis & Anomaly Detection Module:**

Analyzes file access patterns, modification frequency, and unusual activity spikes to identify ransomware-like behavior. Triggers alerts when predefined behavioral thresholds are exceeded.

• **Data Immutability Module (WORM / SnapLock Protection):**

Enforces Write Once Read Many (WORM) policies to prevent unauthorized modification or deletion of critical files. Applies retention policies to ensure data integrity during attacks.

• **Snapshot & Recovery Module:**

Automatically creates and manages point-in-time snapshots of storage volumes. Enables rapid restoration of clean data copies in case of ransomware encryption.

3. Integration Layer:

• Interfaces with storage system services such as FPolicy engine, SnapLock (WORM), and Snapshot manager within the storage environment.

• Integrates with system schedulers for automated snapshot creation.

• Supports integration with external SIEM tools and monitoring systems for centralized logging and threat reporting.

• Allows email or alert notification services for real-time incident reporting.

4. Data Layer:

• Stores file activity logs, anomaly detection records, snapshot metadata, and WORM retention information.

• Maintains historical monitoring data for auditing and forensic analysis.

• Preserves recovery points and system configuration details for rollback operations.

5. Security & Compliance Layer:

• Enforces strict access control policies and role-based permissions for storage administration.

• Ensures compliance with organizational data protection policies and regulatory requirements.

• Protects logs and snapshots from unauthorized access or tampering.

• Implements secure communication protocols for management and monitoring operations.

Flow of Operations:

1. Administrator configures monitoring rules, WORM retention policies, and snapshot schedules through the UI.
2. The File Monitoring module continuously observes file activities in real time.
3. Behavioral Analysis module evaluates activity patterns for anomalies.
4. If suspicious behavior is detected, alerts are generated and WORM protection blocks unauthorized modifications.
5. Snapshots are created periodically, and in case of encryption, clean snapshots are restored to ensure data recovery.

3.2 UML Diagrams

3.2.1 Use Case Diagram

The use case diagram shows a ransomware protection system interacting with three primary actors: the User, the Administrator, and the Attacker. The User accesses files under normal operations, while the Administrator manages security policies, generates logs and reports, and restores snapshots when needed. The Attacker attempts to modify or encrypt files, triggering the system's monitoring process. The system continuously monitors file activity, detects abnormal behavior, enforces file policies, blocks suspicious processes, quarantines affected files, and sends alert notifications. It also integrates with NetApp Storage and an external security engine, illustrating a layered security approach that combines monitoring, detection, prevention, and recovery mechanisms to safeguard data.

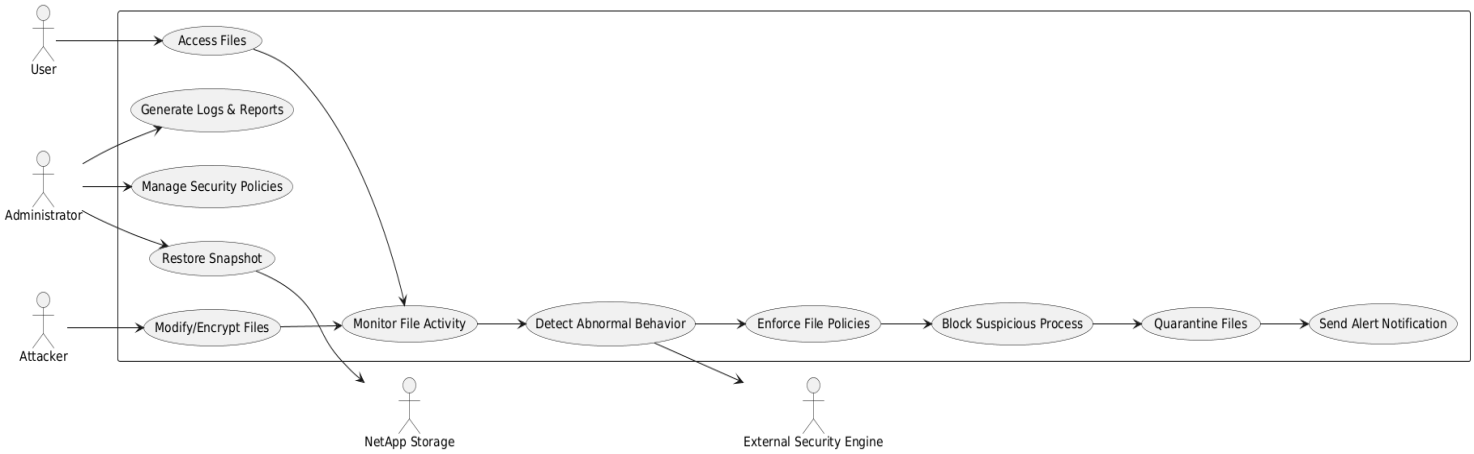


Figure 3.2.1: Use Case Diagram of Ransomware Protection

3.2.2 Class Diagram

Class diagram represents the core components of the ransomware protection system and their interactions. The **FPolicyEngine** class is responsible for monitoring file activity and sending events to the **DetectionEngine**, which analyzes behavior and detects potential ransomware attacks. Once malicious activity is identified, the **PolicyManager** enforces security rules and blocks suspicious processes. From there, multiple supporting services are triggered: the **QuarantineService** isolates infected files, the **NotificationService** sends alerts to administrators, the **SnapshotManager** creates or restores snapshots for recovery, and the **Logger** records events and generates reports. Additionally, the **SnapshotManager** interacts with **WORMStorage**, which protects critical files using immutability mechanisms. Overall, the diagram illustrates a layered, modular architecture that integrates monitoring, detection, enforcement, recovery, logging, and data protection components.

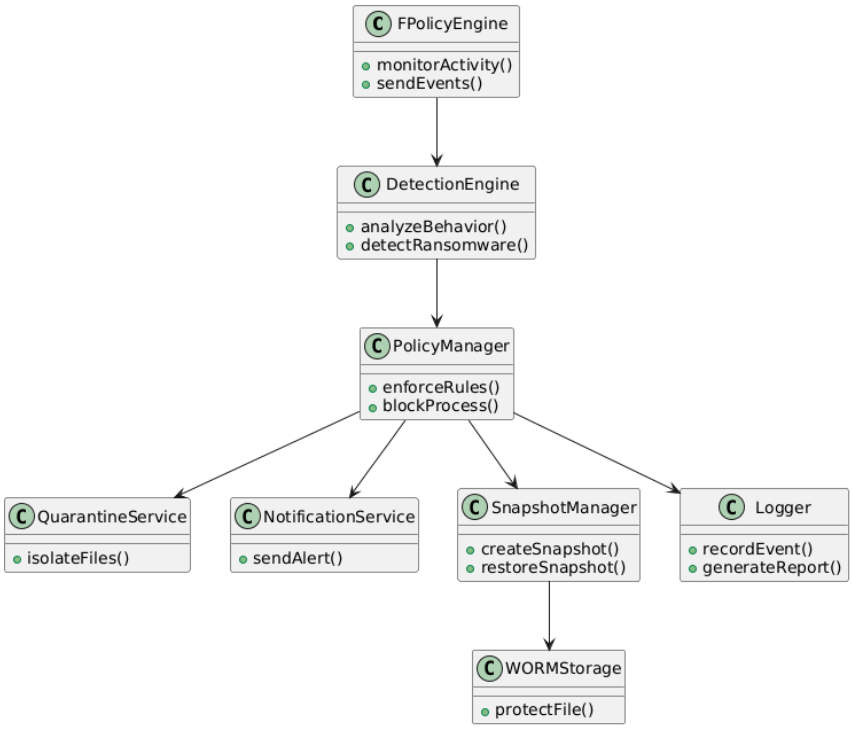


Figure 3.2.2: Class Diagram of Ransomware Protection

3.2.3 Activity Diagram

Activity diagram representis the workflow of the ransomware protection system. The process begins when a user or attacker accesses files, after which the FPolicy module continuously monitors the activity. A decision point checks whether abnormal behavior is detected. If no abnormal activity is found, the system allows normal operation. If suspicious behavior is detected, the system triggers the detection mechanism, blocks the malicious process, quarantines affected files, sends an alert to the administrator, and restores a clean snapshot to recover compromised data. Both paths eventually merge, and the system generates logs and reports before the process ends. This diagram illustrates the step-by-step response mechanism from monitoring to detection, prevention, recovery, and reporting.

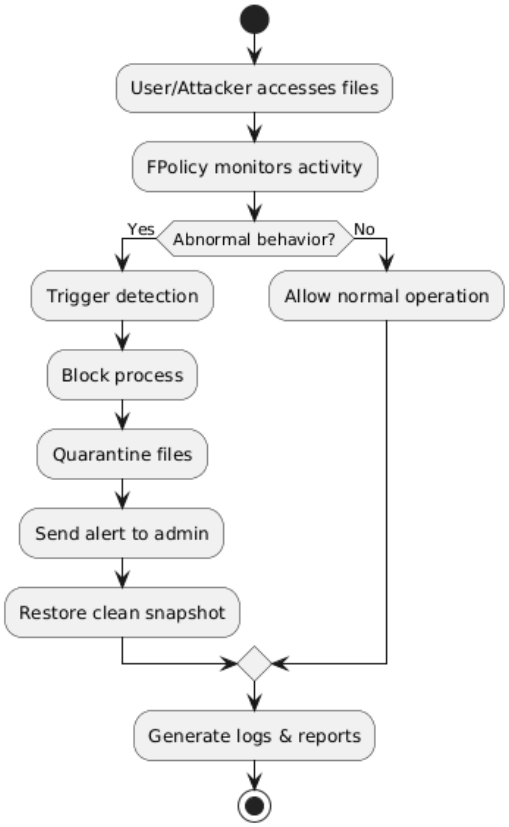


Figure 3.2.3: Activity Diagram of Ransomware Protection

3.2.4 Sequence Diagram

Sequence diagram illustrates the interaction between different components of the ransomware protection system during file access. When a user or attacker initiates file access or mass encryption, the FPolicy Engine captures the activity and sends the data to the Detection Engine for analysis. If suspicious behavior is detected, the flow enters the "Ransomware detected" condition, where the Policy Manager coordinates the response by instructing the Quarantine Service to isolate affected files and the Notification Service to send alerts to the administrator. The administrator may then initiate snapshot restoration from Snapshot Storage to recover clean data. In the alternative "Normal behavior" condition, the system simply allows the operation to continue without intervention. This diagram clearly demonstrates the real-time monitoring, decision-making, and coordinated response workflow across system

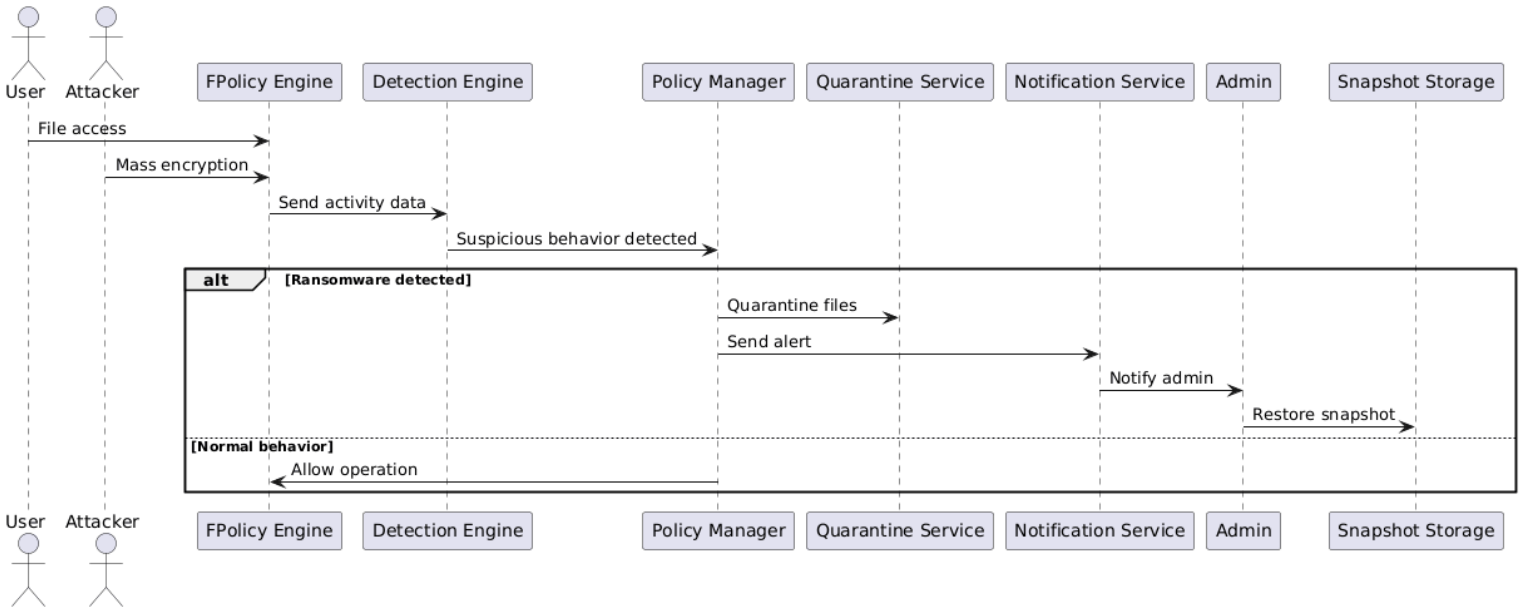


Figure 3.2.4: Sequence Diagram of Ransomware Protection

CHAPTER 4

SYSTEM IMPLEMENTATION

4.1 Modules

1. File Monitoring and Detection
2. Behavioral Analysis & Anomaly Detection
3. Data Immutability (WORM / SnapLock Protection)
4. Snapshot & Recovery

4.2 Module Description

The **File Monitoring and Detection** module continuously observes file system activities such as creation, deletion, renaming, and modification in real time to identify suspicious operations like rapid encryption or mass file changes. The **Behavioral Analysis & Anomaly Detection Module** analyzes file access patterns, modification frequency, and entropy changes to distinguish between normal user behavior and potential ransomware activity, enabling early detection of zero-day attacks. The **Data Immutability Module (WORM / SnapLock Protection)** safeguards critical data by enforcing write-once-read-many policies, preventing unauthorized modification or deletion of protected files during an attack. The **Snapshot & Recovery Module** periodically creates point-in-time snapshots of storage volumes and enables quick restoration of clean data, ensuring minimal downtime and effective recovery in the event of ransomware infection.

4.2.1 File Monitoring and Detection

The **File Monitoring and Detection Module** is responsible for continuously tracking file system activities in real time to identify potentially malicious behavior. It monitors operations such as file creation, deletion, renaming, and modification, especially within network-attached storage environments. By analyzing patterns like rapid file encryption, bulk renaming, unusual extension changes, or high-frequency write operations, the module

can detect early signs of ransomware attacks. Once suspicious activity is identified, it triggers alerts and forwards event data to the detection engine for further analysis, enabling timely response and minimizing the risk of widespread data compromise.

Use case diagram

The use case diagram shows how the administrator interacts with the system to monitor file activities. The system continuously observes file operations such as creation, modification, deletion, and renaming. When suspicious activities such as rapid file encryption or abnormal file changes are detected, the system generates alerts. This diagram mainly highlights the interaction between the admin and the monitoring system.

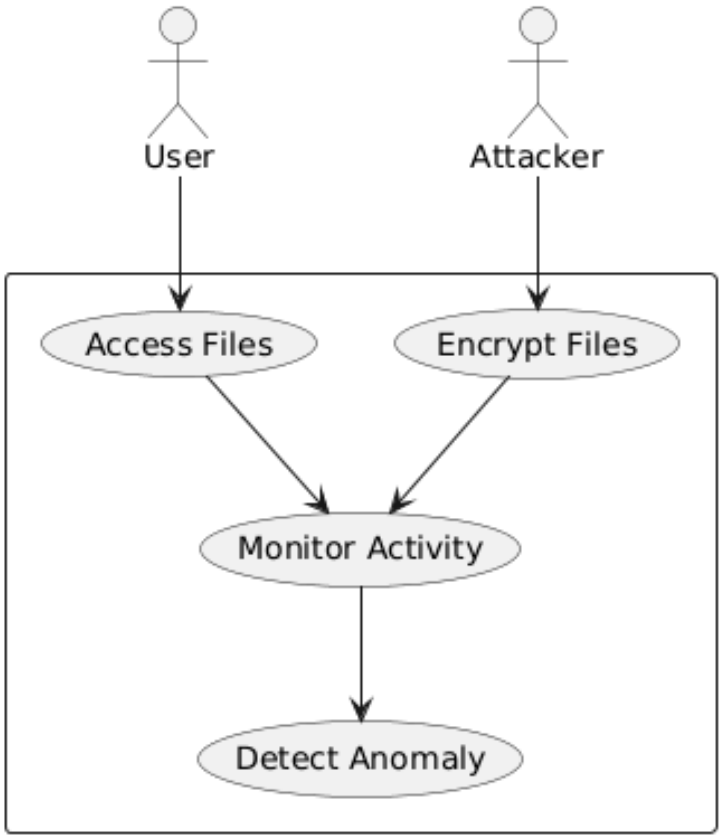


Figure: 4.2.1.1 Use case diagram of File Monitoring and Detection

Class diagram

The class diagram represents the structural design of the file monitoring module. It includes classes such as FileMonitor, FileEvent, and AlertManager. The FileMonitor class is responsible for tracking file activities, while FileEvent stores details about file changes. The AlertManager class handles alert generation and notifications when suspicious activities are detected.

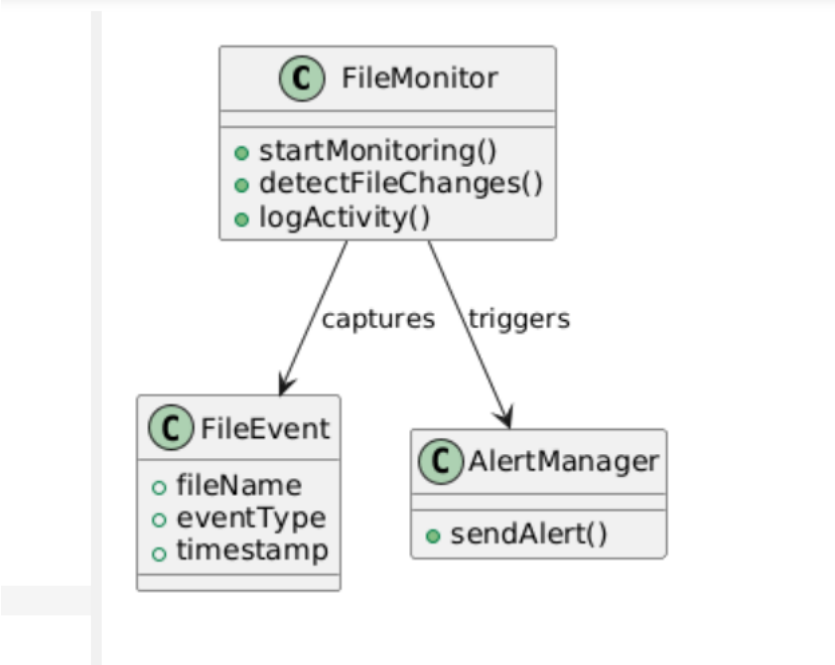


Figure: 4.2.1.2 Class diagram of File Monitoring and Detection

Activity Diagram

The activity diagram illustrates the workflow of the file monitoring process. It begins with the system starting the monitoring service and continuously tracking file operations. When a file change occurs, the system analyzes the activity for suspicious patterns. If abnormal behavior is detected, an alert is generated and logged; otherwise, monitoring continues normally.

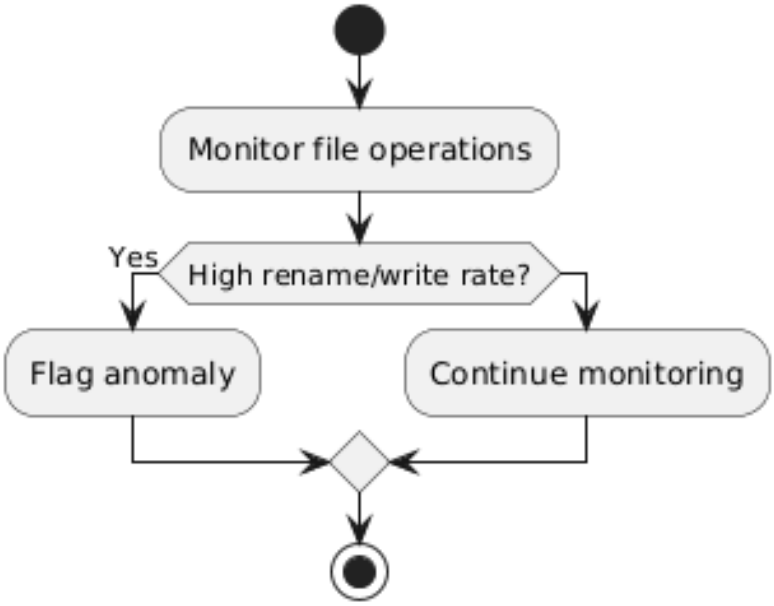


Figure :4.2.1.3 Activity Diagram of File Monitoring and Detection

Sequence Diagram

The sequence diagram shows the interaction between different components during file monitoring. The administrator initiates the monitoring process, and the FileMonitor communicates with the file system to observe file activities. When a suspicious event is detected, the monitoring component sends the information to the AlertManager. The system then notifies the administrator about the detected threat.

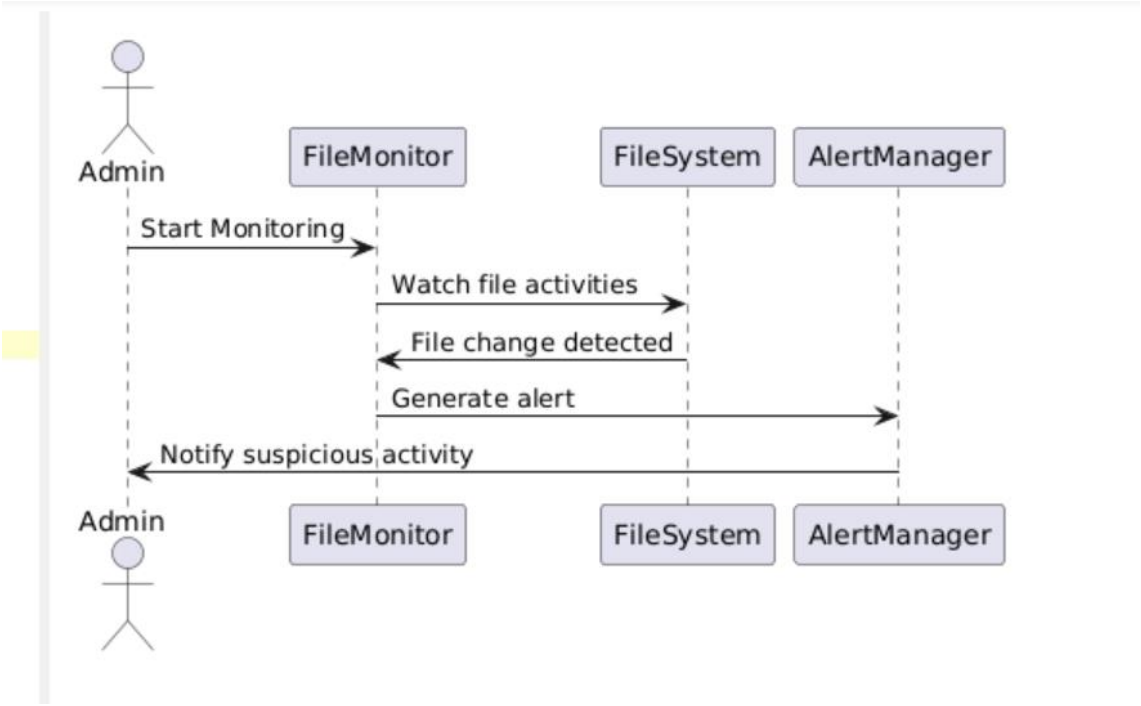


Figure : 4.2.1.4 Sequence Diagram of File Monitoring and Detection

4.2.2 Behaviour Analysis and Anomaly Detection

The **Behavioral Analysis & Anomaly Detection Module** evaluates file access patterns and system behavior to differentiate between normal user activity and potential ransomware actions. It analyzes parameters such as the rate of file modifications, access frequency, unusual spikes in activity, and entropy changes that may indicate encryption. By comparing current behavior against predefined thresholds or baseline patterns, the module identifies deviations that suggest malicious intent. When abnormal behavior is detected, it flags the activity and triggers appropriate security responses, enabling the system to detect zero-day and previously unknown ransomware variants that bypass traditional signature-based methods.

Use case diagram

The use case diagram illustrates how the system analyzes user and file activity patterns to detect anomalies. The administrator can configure monitoring policies and thresholds for abnormal behavior. The system evaluates file access frequency, modification rate, and encryption patterns. If abnormal activity exceeds predefined thresholds, the system flags it as a potential ransomware attack.

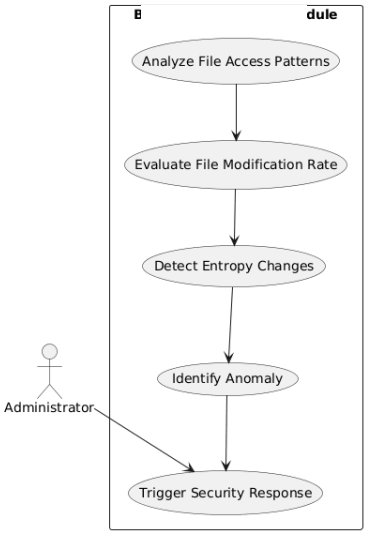


Figure :4.2.2.1 Use case diagram of Behaviour Analysis and Anomaly Detection

Class Diagram

The class diagram represents the main components involved in behavior analysis. It includes classes such as BehaviorAnalyzer, FileActivity, and ThreatDetector. The BehaviorAnalyzer processes activity data collected from the system. The ThreatDetector evaluates whether the behavior indicates a ransomware attack based on predefined thresholds.

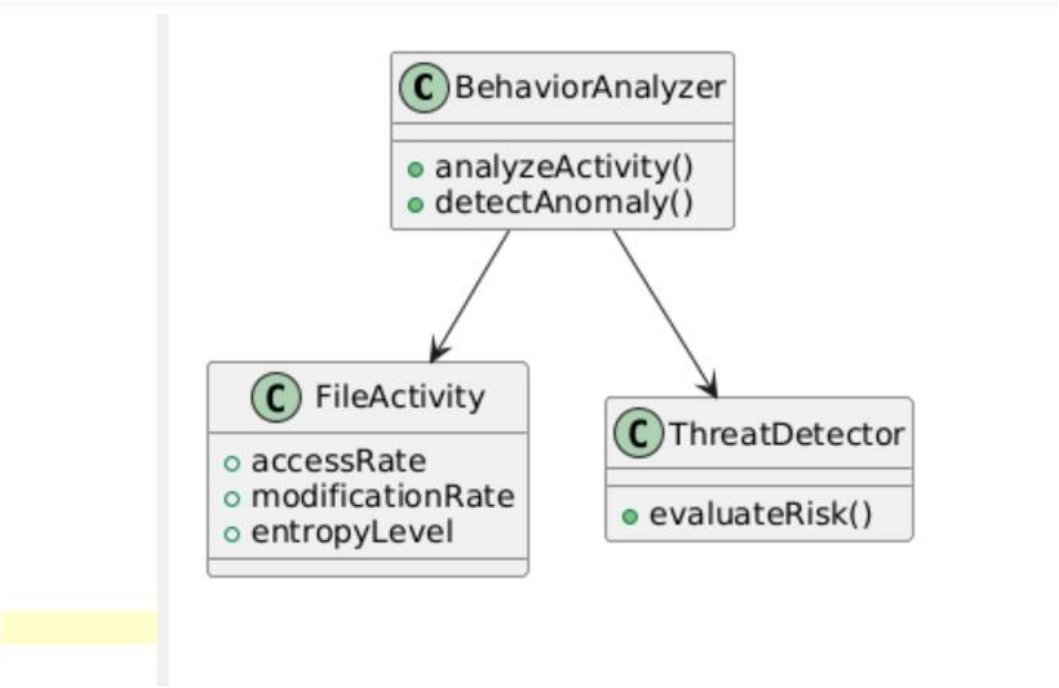


Figure: 4.2.2.2 Class Diagram of Behaviour Analysis and Anomaly Detection

Activity Diagram

The activity diagram describes the process of analyzing file activity behavior. It starts with collecting file activity data from the monitoring module. The system then evaluates patterns such as file access rate and entropy changes. If suspicious behavior is detected, the system marks it as a threat and generates alerts; otherwise, it continues monitoring normal activities.

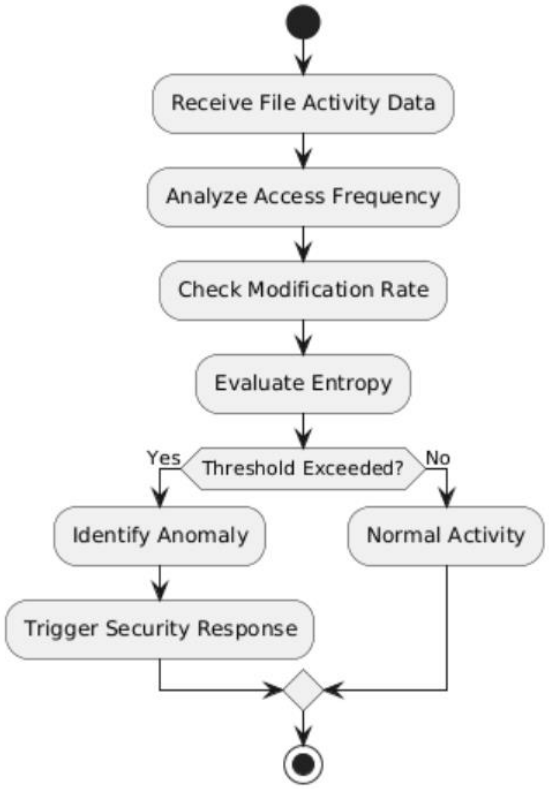


Figure: 4.2.2.3 Activity Diagram of Behaviour Analysis and Anomaly

Sequence Diagram

The sequence diagram demonstrates the flow of data during anomaly detection. The system sends file activity data to the BehaviorAnalyzer for evaluation. The analyzer processes the data and forwards it to the ThreatDetector. If the activity is considered suspicious, the system flags it as a potential ransomware threat and alerts the administrator.

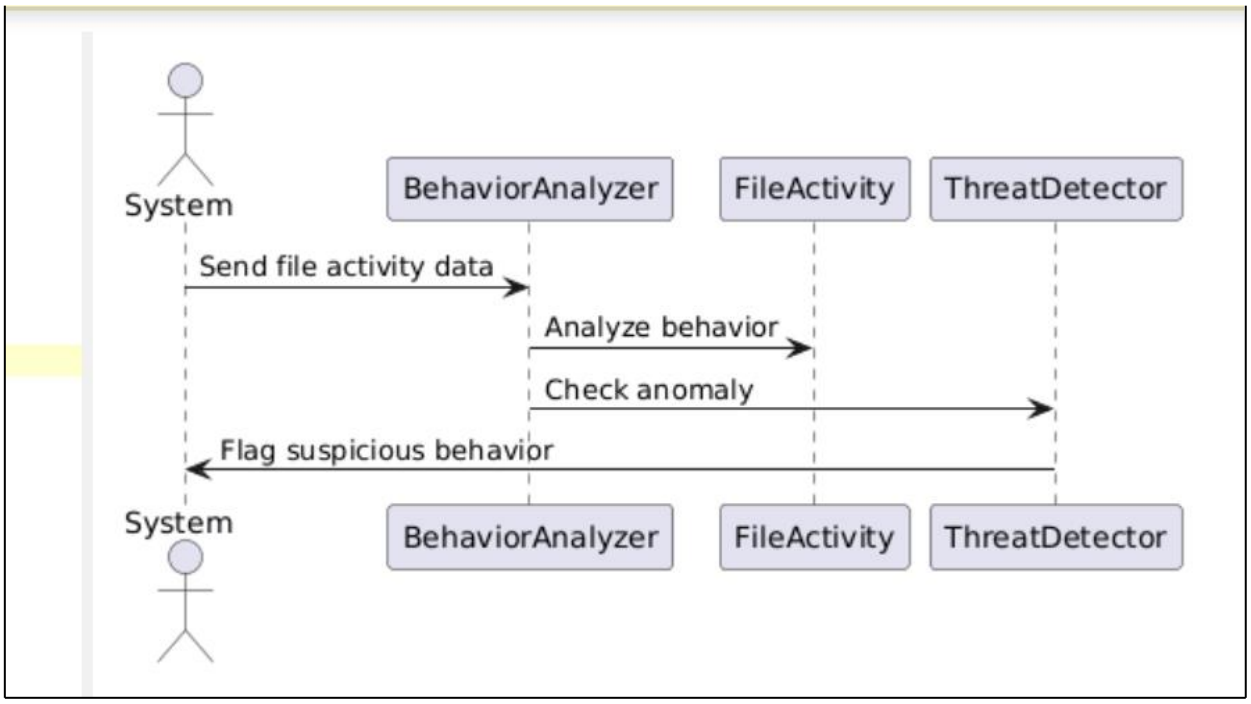


Figure: 4.2.2.4 Sequence Diagram of Behaviour Analysis and Anomaly Detection

4.2.3 The Data Immutability

The **Data Immutability Module (WORM / SnapLock Protection)** ensures that critical data remains secure and tamper-proof by enforcing Write Once Read Many (WORM) policies. Once files are written and committed under a defined retention period, they cannot be modified, overwritten, or deleted until the retention expires. This prevents ransomware or unauthorized users from encrypting or altering protected data. By applying strict retention rules and access controls, the module maintains data integrity, supports compliance requirements, and acts as a strong defensive layer even during active cyberattacks.

Use case diagram

The use case diagram shows how administrators configure immutability policies to protect critical files. The system enables WORM (Write Once Read Many) protection to prevent modification or deletion of important data. Once files are locked with retention policies, ransomware cannot encrypt or alter them. This ensures data integrity and compliance with security policies.

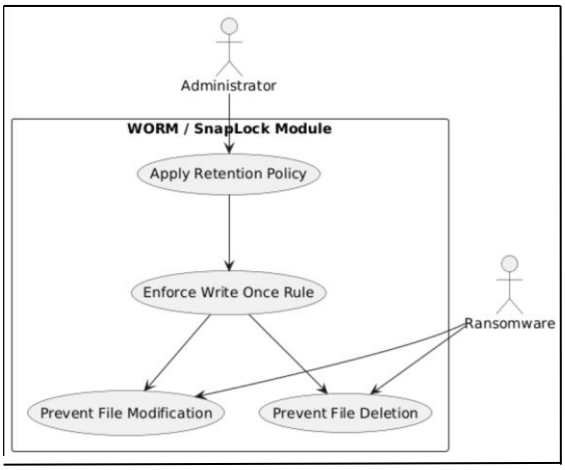


Figure:4.2.3.1 Use case diagram of The Data Immutability

Class Diagram

The class diagram describes the structural components of the immutability module. It includes classes such as WORMController, SnapLockManager, and ProtectedFile. The WORMController manages immutability settings and retention policies. The SnapLockManager ensures that protected files remain locked and tamper-proof.

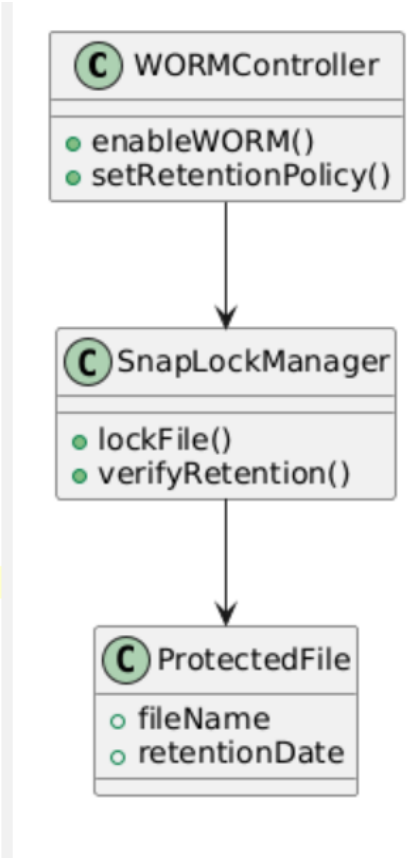


Figure: 4.2.3.2 Class Diagram of The Data Immutability

Activity Diagram

The activity diagram explains the workflow of enabling immutability protection. The administrator activates WORM or SnapLock policies on selected storage volumes. The system applies retention policies to protect files. Once the data is locked, it cannot be modified or deleted until the retention period expires.

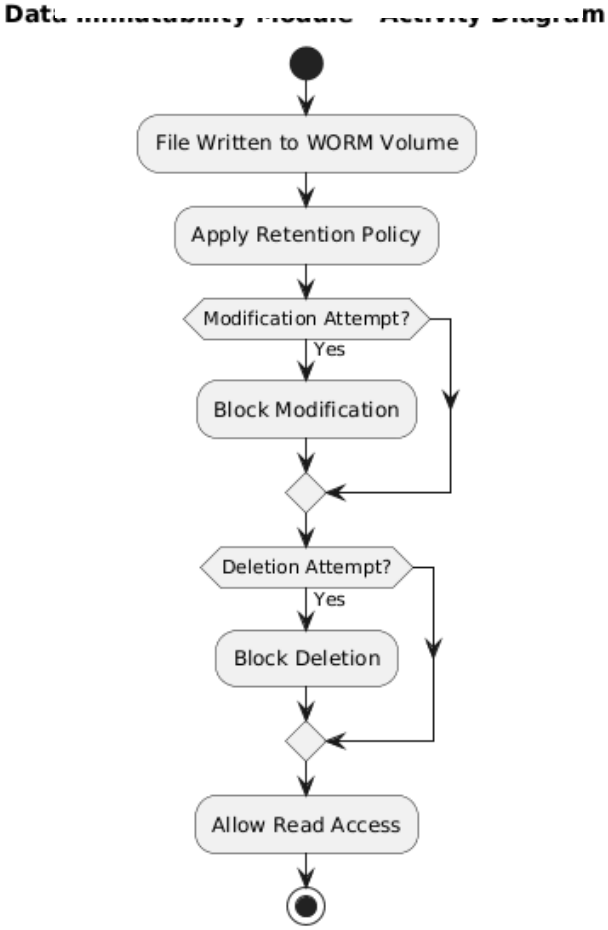


Figure:4.2.3.3 Activity Diagram of The Data Immutability

Sequence Diagram

The sequence diagram shows the process of enabling immutability protection. The administrator initiates the WORM protection request. The WORMController communicates with the SnapLockManager to apply retention policies. The SnapLockManager then locks the file and confirms that the data is protected from modification.

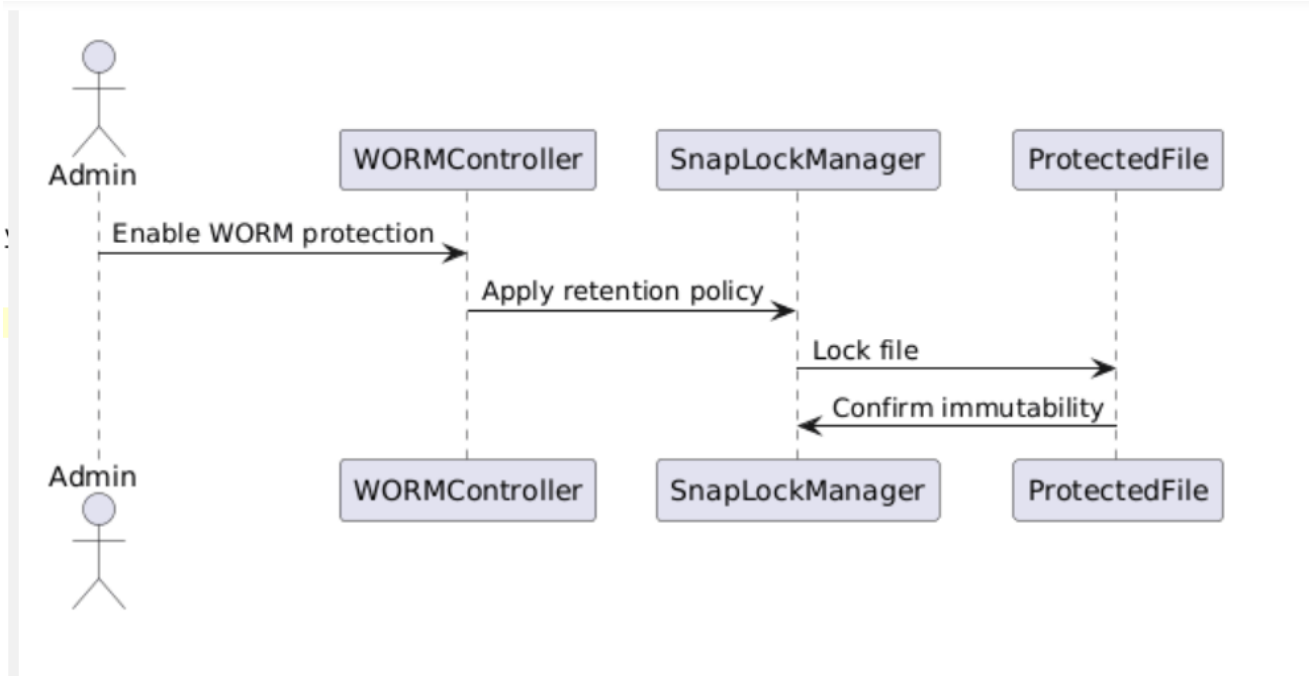


Figure:4.2.3.4 Sequence Diagram of The Data Immutability

4.2.4 The Snapshot and Recovery

The **Snapshot & Recovery Module** is designed to ensure rapid data restoration and business continuity in the event of a ransomware attack. It automatically creates periodic point-in-time snapshots of storage volumes without duplicating the entire dataset, making the process storage-efficient. If files are encrypted or corrupted, administrators can quickly roll back the system to a clean snapshot taken before the attack occurred. This minimizes downtime, reduces data loss, and provides a reliable recovery mechanism, serving as the final layer of defense in the multi-layered ransomware protection architecture.

Use case diagram

The use case diagram illustrates how administrators manage snapshots and recovery operations. The system periodically creates snapshots of storage volumes. In case of a ransomware attack, administrators can restore files from a previous clean snapshot. This ensures quick recovery and reduces data loss.

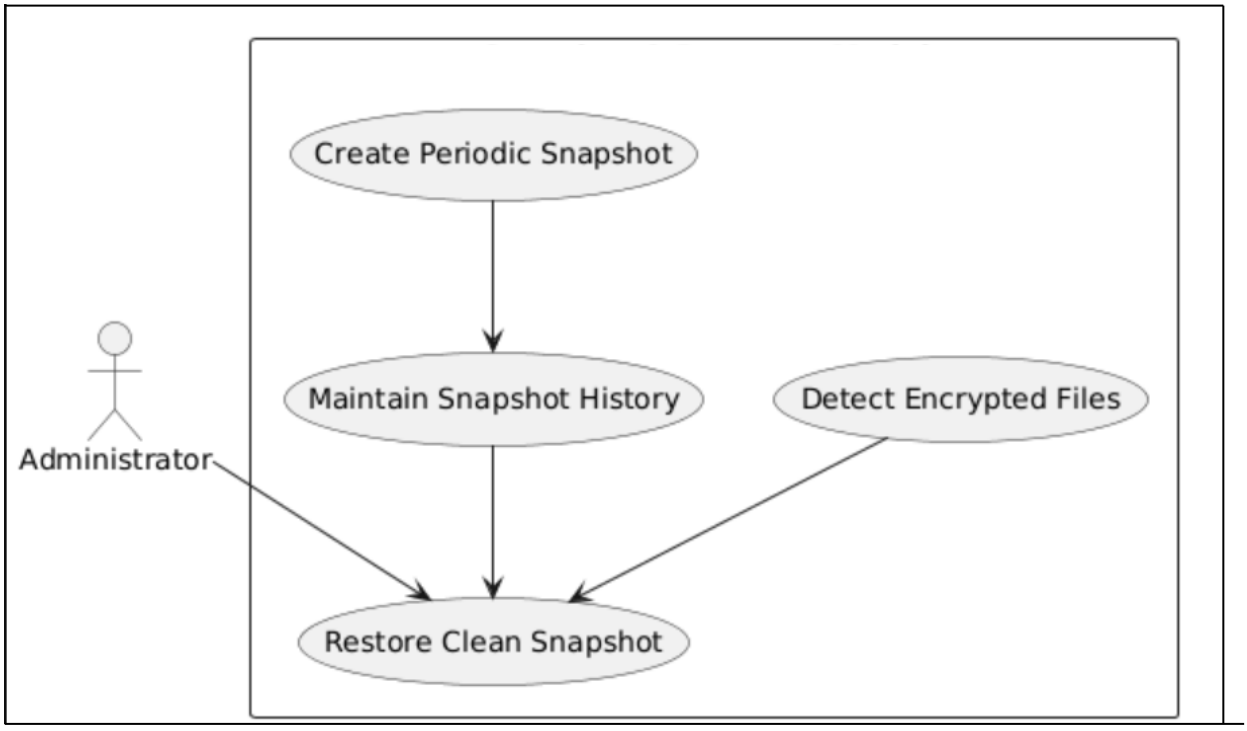


Figure:4.2.4.1 Use case diagram of The Snapshot and Recovery

Class Diagram

The class diagram shows the structural design of the snapshot management system. It includes classes such as SnapshotManager, RecoveryManager, and StorageVolume. The SnapshotManager handles snapshot creation and scheduling. The RecoveryManager is responsible for restoring data from stored snapshots.

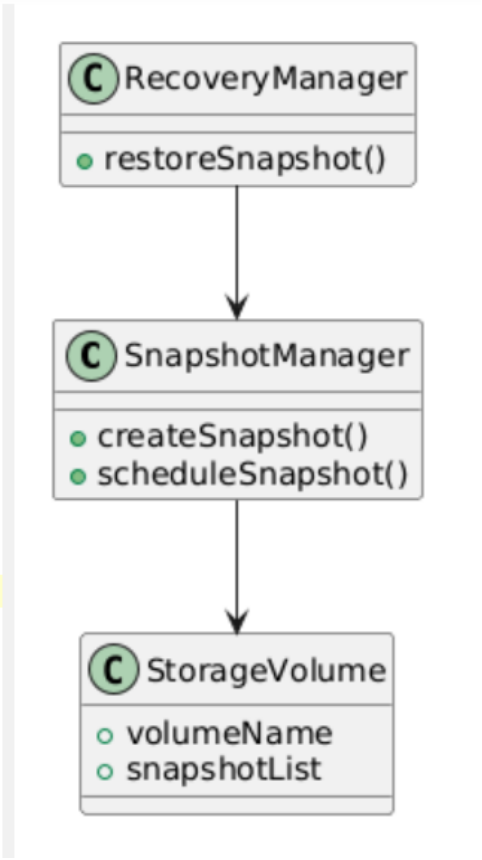


Figure: 4.2.4.2 Class Diagram of The Snapshot and Recovery

Activity Diagram

The activity diagram represents the process of snapshot creation and data recovery. The system schedules and creates snapshots at regular intervals. If ransomware activity is detected, the administrator selects a clean snapshot for recovery. The system restores the data and resumes normal operations.

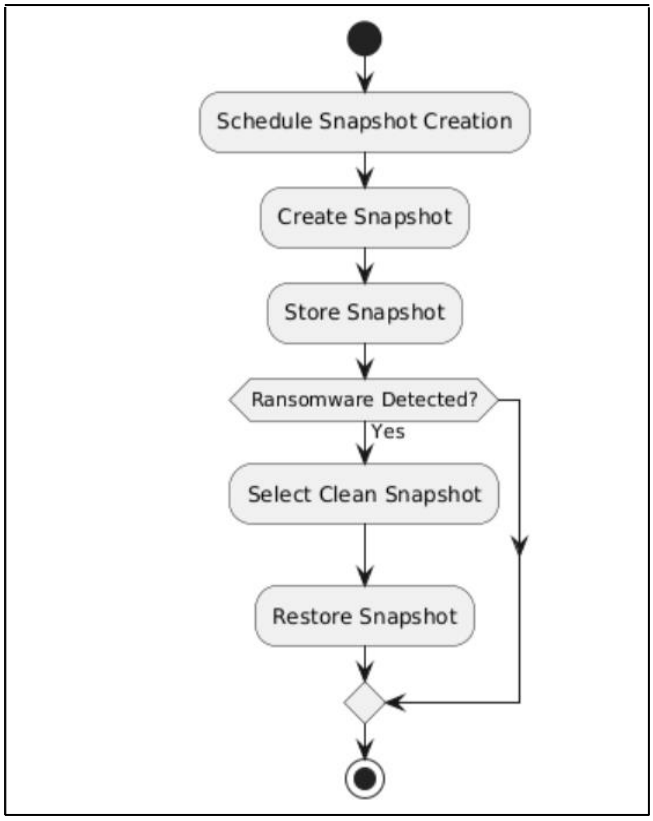


Figure:4.2.4.3 Activity Diagram of The Snapshot and Recovery

Sequence Diagram

The sequence diagram explains how snapshots are created and used for recovery. The administrator requests the creation of a snapshot, and the SnapshotManager stores it in the storage volume. When recovery is required, the administrator requests restoration. The RecoveryManager retrieves the snapshot and restores the data.

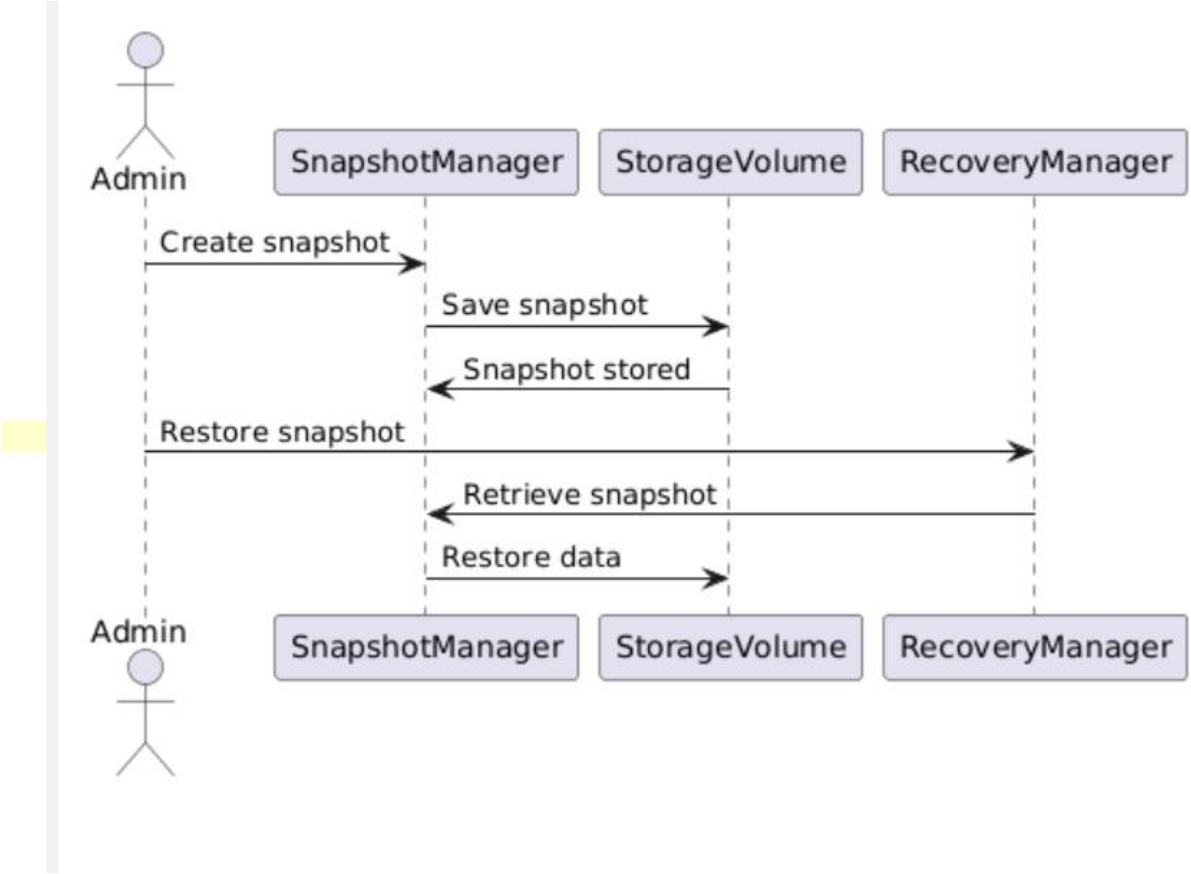


Figure:4.2.4.4 Sequence Diagram of The Snapshot and Recovery

4.3 ALGORITHM

The algorithm of the ransomware protection system begins by continuously monitoring **file activities** such as **creation, modification, deletion, and renaming** in **real time**. For each detected event, the system analyzes **behavioral parameters** including **frequency of file changes, rate of access, entropy variation**, and **deviation from normal usage patterns**. It establishes a **baseline of normal behavior** and continuously compares incoming activities against this baseline to improve **detection accuracy**. If the calculated **behavior score** exceeds a predefined **anomaly threshold**, the system flags the activity as **suspicious**, immediately **blocks the process** through **policy enforcement**, **quarantines affected files**, and **sends alerts** to the administrator. Simultaneously, critical data protected under **WORM/SnapLock policies** remains **immutable** and cannot be altered, ensuring **data integrity** during attacks. The system also **isolates affected storage segments** to prevent **lateral spread** of ransomware. In case of confirmed compromise, the system initiates **restoration from the latest clean snapshot** to recover data and resume **normal operations**. Additionally, all **events, alerts, and recovery actions** are **logged and analyzed** to improve future detection capabilities and support **auditing, reporting, and forensic analysis**.

CHAPTER 5

Testing

5.1 Unit Testing

Unit Testing for this project focuses on validating each module independently to ensure reliable ransomware protection functionality. The File Monitoring module is tested by simulating file creation, deletion, renaming, and modification events to verify accurate detection and logging. The Behavioral Analysis module is tested using abnormal file access patterns and encryption-like activity to confirm correct anomaly detection. The Data Immutability module is tested by attempting unauthorized modification or deletion of WORM-protected files to ensure enforcement works correctly. The Snapshot & Recovery module is tested by creating test snapshots and restoring sample data to validate recovery accuracy. Each unit is verified in isolation using controlled inputs to ensure it performs its intended function correctly before system integration.

5.1.1 Test Cases

Test Case ID	Module	Test Description	Input Data	Expected Output
TC-01	File Monitoring & Detection	Verify detection of mass file modification	Multiple files modified within short time	System detects abnormal activity and raises alert
TC-02	File Monitoring & Detection	Detect rapid file renaming (ransomware behavior)	Bulk file rename operation	Suspicious activity flagged and logged
TC-03	File Monitoring & Detection	Detect unauthorized file deletion	Delete protected files	Alert generated and deletion event recorded
TC-04	Behavioral Analysis & Anomaly Detection	Identify abnormal file access rate	High-frequency file access	Behavior score exceeds threshold and triggers warning
TC-05	Behavioral Analysis & Anomaly Detection	Detect entropy increase in files	Encrypted file patterns	System flags potential encryption activity
TC-06	Behavioral Analysis & Anomaly Detection	Identify deviation from normal user behavior	Unusual file operations	Anomaly alert generated
TC-07	Data Immutability (WORM / SnapLock)	Prevent modification of protected files	Attempt to edit WORM-protected file	Modification denied
TC-08	Data Immutability (WORM / SnapLock)	Prevent deletion of immutable files	Attempt to delete protected file	Deletion blocked successfully

Test Case ID	Module	Test Description	Input Data	Expected Output
TC-09	Snapshot & Recovery	Verify snapshot creation	Trigger snapshot schedule	Snapshot created successfully
TC-10	Snapshot & Recovery	Restore data after simulated attack	Encrypted / modified files	Data restored from latest clean snapshot

Table 5.1 Test Cases

5.2 Integration Testing

Integration Testing for this project ensures that all modules work together seamlessly to provide end-to-end ransomware protection. The File Monitoring module is integrated with the Behavioral Analysis module to verify that detected file changes are correctly analyzed for suspicious activity. The Behavioral Analysis module is tested alongside the Data Immutability module to ensure that when anomalies are detected, critical files can be automatically protected using WORM/SnapLock mechanisms. The Snapshot & Recovery module is integrated with all other modules to confirm that, upon detection of potential ransomware behavior, snapshots are triggered and data restoration can be performed without conflict. These tests validate smooth communication between modules, proper data flow, and coordinated response to threats across the entire system.

5.3 System Testing

System Testing for this project validates the complete ransomware protection solution as a fully integrated system to ensure it meets functional and security requirements. It tests whether the File Monitoring and Detection module successfully identifies suspicious file activities, the Behavioral Analysis module accurately detects anomalies, the Data Immutability module effectively protects critical data using WORM/SnapLock mechanisms, and the Snapshot & Recovery module reliably restores files after simulated ransomware attacks. The entire workflow is tested under real-world scenarios to confirm that threat detection, automatic protection, alert generation, and data recovery operate correctly without failures, ensuring the system delivers a robust and coordinated defense against ransomware threats.and efficiently within the target Linux environment.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 RESULTS

The ransomware protection system was successfully developed and tested to provide automated detection, prevention, and recovery mechanisms against ransomware threats. The File Monitoring and Detection module efficiently tracked file operations such as creation, modification, deletion, and renaming in real time. During testing, the system accurately identified suspicious activities such as rapid file changes, abnormal extensions, and high-frequency write operations commonly associated with ransomware behavior, enabling early-stage threat detection without affecting normal system performance.

The Behavioral Analysis and Anomaly Detection module effectively distinguished between legitimate user actions and malicious encryption patterns by analyzing file access rates, entropy variations, and usage deviations. The system consistently flagged abnormal activity when predefined thresholds were exceeded, proving its capability to detect both known and zero-day ransomware attacks. The Data Immutability module successfully enforced WORM-based protection, ensuring that critical files remained tamper-proof and inaccessible to unauthorized modification or deletion during simulated attack scenarios.

The Snapshot and Recovery module demonstrated reliable performance by creating periodic point-in-time backups and restoring compromised data quickly after simulated ransomware encryption. Recovery was completed without data loss, ensuring system continuity. Overall, the results confirm that the system provides a robust, automated, and multi-layered defense mechanism capable of detecting, preventing, and recovering from ransomware attacks while maintaining data integrity and availability.

6.2 DISCUSSION

The development and implementation of this ransomware protection system highlight the increasing need for proactive and automated defense mechanisms in modern storage environments. Traditional security solutions often rely on signature-based detection, which may fail against evolving ransomware variants. This project demonstrates that integrating real-time monitoring, behavior-based detection, data immutability, and snapshot recovery can provide a more resilient and comprehensive protection strategy.

During evaluation, the system showed strong effectiveness in detecting suspicious file activity and abnormal behavior patterns without generating excessive false positives. The File Monitoring and Behavioral Analysis modules worked together to provide early threat detection, reducing the potential impact of ransomware attacks. The WORM-based immutability mechanism proved crucial in safeguarding sensitive data by preventing unauthorized encryption or deletion, while the Snapshot and Recovery module ensured rapid restoration of affected files.

The findings confirm that a layered protection approach significantly enhances system resilience against ransomware threats. By combining detection, prevention, and recovery capabilities into a unified framework, the project presents a scalable and practical solution suitable for enterprise storage systems and critical data environments. The discussion emphasizes that such automated protection systems can play a vital role in strengthening cybersecurity readiness and ensuring business continuity in the face of advanced ransomware attacks.

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 CONCLUSION

The Ransomware Protection System provides an effective and intelligent approach to safeguarding data against modern ransomware threats through automation and layered defense mechanisms. By integrating key modules such as File Monitoring and Detection, Behavioral Analysis, Data Immutability (WORM), and Snapshot-based Recovery, the system ensures that ransomware activities are detected early, prevented from causing permanent damage, and mitigated through rapid recovery processes. This approach minimizes reliance on manual intervention and enables consistent protection of critical data assets.

Testing demonstrated that the system could successfully monitor file activities in real time, identify abnormal behavioral patterns, enforce tamper-proof data protection using immutability techniques, and restore compromised data efficiently through snapshots. The system operated reliably without affecting normal storage operations, ensuring both security and performance stability. Its automated detection and recovery capabilities significantly reduced the impact of simulated ransomware attacks.

Overall, the project validates that combining real-time monitoring, behavior-based detection, immutable storage, and snapshot recovery provides a strong and practical defense against ransomware. The system enhances data resilience, ensures integrity, and supports business continuity. It also establishes a solid foundation for future advancements in intelligent ransomware defense mechanisms for enterprise storage environments.

7.2 Future Enhancements

The Ransomware Protection System can be further enhanced to improve its intelligence, scalability, and adaptability in evolving threat landscapes. One major enhancement could be the integration of AI or machine learning-based detection models to identify advanced ransomware behaviors and previously unseen attack patterns beyond predefined thresholds. This would improve detection accuracy and reduce false positives.

Another potential improvement is the development of a centralized dashboard or web-based interface that enables administrators to visualize file activity, threat alerts, snapshot status, and recovery operations in real time. Integration with Security Information and Event Management (SIEM) systems could provide centralized monitoring and faster incident response. Automated alert mechanisms such as email or SMS notifications can also be implemented to instantly inform administrators about suspicious activities or recovery actions.

Future versions may include support for hybrid and cloud storage environments, enabling protection across distributed infrastructures. Additionally, integrating predictive analytics for risk assessment, automated retention policy management, and adaptive snapshot scheduling could further strengthen data protection. These enhancements would transform the system into a more intelligent, scalable, and enterprise-ready ransomware defense solution capable of addressing next-generation cyber threats.

REFERENCE

[1] Furnell, S., Emm, D. “The ABC of Ransomware Protection”, *Computer Fraud & Security*, 2017, vol. 2017, no. 10, pp. 5–11.

[2] Microsoft. “Microsoft Security Bulletin MS17-010 – Critical Security Update for Microsoft Windows SMB Server”, *Microsoft Security Bulletin*, 2017, vol. 17, no. 1, pp. 1–8.

[3] Loscocco, P., Smalley, S. “Security-Enhanced Linux: Lessons in Mandatory Access Control”, *IEEE Transactions on Secure Computing*, 2005, vol. 2, no. 1, pp. 34–45.

[4] Ivanov, A., Emm, D., Sinitsyn, F., Pontiroli, S. “Kaspersky Security Bulletin 2016: The Ransomware Revolution”, *Securelist Security Report*, 2016, vol. 8, no. 2, pp. 1–15.

[5] National Cyber Security Centre. “Ransomware: Latest Guidance”, *NCSC Security Report*, 2017, vol. 5, no. 1, pp. 10–18.

[6] Zekrifa, D. M. S., Ghorbani, A. “Ameliorate Competitive Learning Neural Networks for System Intrusion Detection”, *Neurocomputing*, 2013, vol. 140, no. 1, pp. 135–145.

[7] Sinitsyn, F. “Locky: The Encryptor Taking the World by Storm”, *Securelist Threat Research*, 2016, vol. 4, no. 2, pp. 22–30.

[8] Goodin, D. “WannaCry Ransomware Spread Widely Because of Windows 7, Not XP”, *Ars Technica Security Review*, 2017, vol. 9, no. 3, pp. 40–46.

[9] Securelist. “Schroedinger’s Pet(ya)”, *Securelist Threat Intelligence Report*, 2017, vol. 6, no. 2, pp. 12–20.

APPENDIX I – SOURCE CODE

```
#!/bin/bash

############################################################
####
# ALL-IN-ONE RANSOMWARE SIMULATION & RECOVERY DEMO
# NetApp ONTAP + FPolicy + SnapLock + Snapshot
# Academic Project Version
############################################################
####

############################
# USER CONFIGURATION
############################

ONTAP_IP="10.2.2.20"
ONTAP_USER="dharsan"

SVM_NAME="svm01"
VOL_NAME="volnfs"
SNAP_NAME="pre_attack_snapshot"

TARGET_DIR="/.demo_data"
WORM_DIR="/worm_data"
THREADS=5
FILE_COUNT=10
```

```
LOGFILE="ransomware_full_demo.log"
FPOLICY_DIR="/fpolicy_data"
############################

RED='\033[1;31m'
GREEN='\033[1;32m'
YELLOW='\033[1;33m'
BLUE='\033[1;34m'
NC='\033[0m'

pause(){
   read -p "Press Enter to continue..."
}

run_ontap(){
   ssh ${ONTAP_USER}@${ONTAP_IP} "$1" >> $LOGFILE 2>&1
}

generate_data(){

echo -e "${YELLOW}Generating $FILE_COUNT test files...${NC}"

mkdir -p $TARGET_DIR
rm -rf $TARGET_DIR/*

for i in $(seq 1 $FILE_COUNT)
do
```

```
 echo "Sensitive Data File $i" > $TARGET_DIR/file$i.txt
done

echo -e "${GREEN}Test Data Generated Successfully.${NC}"
}

create_worm(){


echo -e "${YELLOW}Generating $FILE_COUNT in WORM test files...${NC}"



for i in $(seq 1 $FILE_COUNT)
do
   echo "Sensitive Data File $i" > $WORM_DIR/file_$i.txt
done

echo -e "${GREEN}Test Data Generated Successfully.${NC}"
#run_ontap "file fingerprint start -vserver svm01 -file /vol/vol_worm/file1.txt"

}

worm_attack() {
echo -e "${RED}Starting Multi-Threaded Ransomware Attack...${NC}"
```

```
START=$(date +%s)

find "$WORM_DIR" -type f -name "*.txt" | \
xargs -P $THREADS -I {} bash -c 'encrypt_file "$@"' _ {}

END=$(date +%s)
DURATION=$((END - START))

echo "Attack completed in $DURATION seconds" >> $LOGFILE

cat <<EOF > "$WORM_DIR/READ_ME_RESTORE_FILES.txt"
=========================================
YOUR FILES HAVE BEEN ENCRYPTED!
This is an academic ransomware demo.
Recovery possible via NetApp Snapshot.
=========================================
EOF

echo -e "${GREEN}Attack Simulation Completed.${NC}"
echo -e "${BLUE}Total Time: $DURATION seconds${NC}"
}
take_snapshot(){

echo -e "${BLUE}Taking Pre-Attack Snapshot...${NC}"

run_ontap "snapshot create -vserver $SVM_NAME -volume $VOL_NAME -snapshot $SNAP_NAME"
```

```
echo -e "${GREEN}Snapshot Created in ONTAP.${NC}"
}


fpolicy_attack(){

echo -e "${YELLOW}Generating $FILE_COUNT test files...${NC}"

for i in $(seq 1 $FILE_COUNT)
do
   echo "Sensitive Data File $i" > $FPOLICY_DIR/file$i.txt
done

echo -e "${GREEN}Test Data Generated Successfully.${NC}"
}

############################
encrypt_file() {
   file="$1"
   if [ -f "$file" ]; then
      base64 "$file" > "$file.enc"
      rm "$file"
      echo "Encrypted: $file" >> $LOGFILE
   fi
}

export -f encrypt_file
```

```
export LOGFILE

simulate_attack(){

echo -e "${RED}Starting Multi-Threaded Ransomware Attack...${NC}"
START=$(date +%s)

find "$TARGET_DIR" -type f -name "*.txt" | \
xargs -P $THREADS -I {} bash -c 'encrypt_file "$@"' _ {}

END=$(date +%s)
DURATION=$((END - START))

echo "Attack completed in $DURATION seconds" >> $LOGFILE

cat <<EOF > "$TARGET_DIR/READ_ME_RESTORE_FILES.txt"
========================================
YOUR FILES HAVE BEEN ENCRYPTED!
This is an academic ransomware demo.
Recovery possible via NetApp Snapshot.
========================================
EOF

echo -e "${GREEN}Attack Simulation Completed.${NC}"
echo -e "${BLUE}Total Time: $DURATION seconds${NC}"
}
```

```
fpolicy_attack01(){

echo -e "${RED}Starting Multi-Threaded Ransomware Attack...${NC}"
START=$(date +%s)

find "$FPOLICY_DIR" -type f -name "*.txt" | \
xargs -P $THREADS -I {} bash -c 'encrypt_file "$@"' _ {}

END=$(date +%s)
DURATION=$((END - START))

echo "Attack completed in $DURATION seconds" >> $LOGFILE

cat <<EOF > "$FPOLICY_DIR/READ_ME_RESTORE_FILES.txt"
========================================
YOUR FILES HAVE BEEN ENCRYPTED!
This is an academic ransomware demo.
Recovery possible via NetApp Snapshot.
========================================
EOF

echo -e "${GREEN}Attack Simulation Completed.${NC}"
echo -e "${BLUE}Total Time: $DURATION seconds${NC}"
}
restore_snapshot(){

echo -e "${YELLOW}Restoring from NetApp Snapshot...${NC}"
```

```
run_ontap "snapshot restore -vserver $SVM_NAME -volume $VOL_NAME -snapshot $SNAP_NAME"

echo -e "${GREEN}Instant Recovery Completed.${NC}"
}

restore_local(){

echo -e "${YELLOW}Restoring Local Files...${NC}"

find "$TARGET_DIR" -type f -name "*.enc" | while read file
do
   original="${file%.enc}"
   base64 -d "$file" > "$original"
   rm "$file"
done

rm -f "$TARGET_DIR/READ_ME_RESTORE_FILES.txt"

echo -e "${GREEN}Local Files Restored Successfully.${NC}"
}

generate_report(){

echo "==================================="
echo " RANSOMWARE ATTACK REPORT"
echo "==================================="
echo "Target Directory : $TARGET_DIR"
echo "Total Files      : $FILE_COUNT"
```

```
echo "Threads Used     : $THREADS"
echo "Log File         : $LOGFILE"
echo "==================================="
echo "Attack Log Preview:"
tail -10 $LOGFILE
echo "==================================="
}

############################
# MAIN MENU
############################

while true
do
clear
echo -e "${GREEN}=========================================${NC}"
echo -e "${GREEN}  FULL RANSOMWARE DETECTION & RECOVERY SYSTEM${NC}"
echo -e "${GREEN}=========================================${NC}"
echo "1. Generate Test Data"
echo "2. Take Pre-Attack Snapshot (ONTAP)"
echo "3. Start Multi-Threaded Ransomware Attack"
echo "4. Restore from ONTAP Snapshot (Instant Recovery)"
echo "5. Generate Attack Report"
echo "6. Create a WORM files"
echo "7. WORM Files Attack"
```

```
echo "8. Exit"
echo
"================================================"
read -p "Enter your choice: " choice

case $choice in
    1) generate_data; pause ;;
    2) take_snapshot; pause ;;
    3) simulate_attack; pause ;;
    4) restore_snapshot; pause ;;
    5) generate_report; pause ;;
    6) create_worm;  pause ;;
##    7) fpolicy_attack; fpolicy_attack01; pause;;
    7) worm_attack; pause;;
    8) exit 0 ;;
       *) echo -e "${RED}Invalid Option${NC}"; pause ;;
esac

done
```

APPENDIX II – SCREENSHOTS

```
DXC-CLUSTER::> vserver fpolicy policy create -vserver svm-senthil -policy-name r
ansom_block -events fp-event1 -engine native -is-mandatory true

DXC-CLUSTER::> vserver fpolicy policy show
  Vserver       Policy       Events     Engine         Is Mandatory  Privileged
                Name                                                 Access
--------------- -----------  ---------- -------------  ------------  -----------
svm-senthil     ransom_      fp-event1  native         true          no
                block
```

Creation of FPolicy

```
DXC-CLUSTER::> vserver fpolicy enable -vserver svm-senthil -policy-name ransom_b
lock -sequence-number 1

DXC-CLUSTER::> vserver fploicy show

Error: "fploicy" is not a recognized command

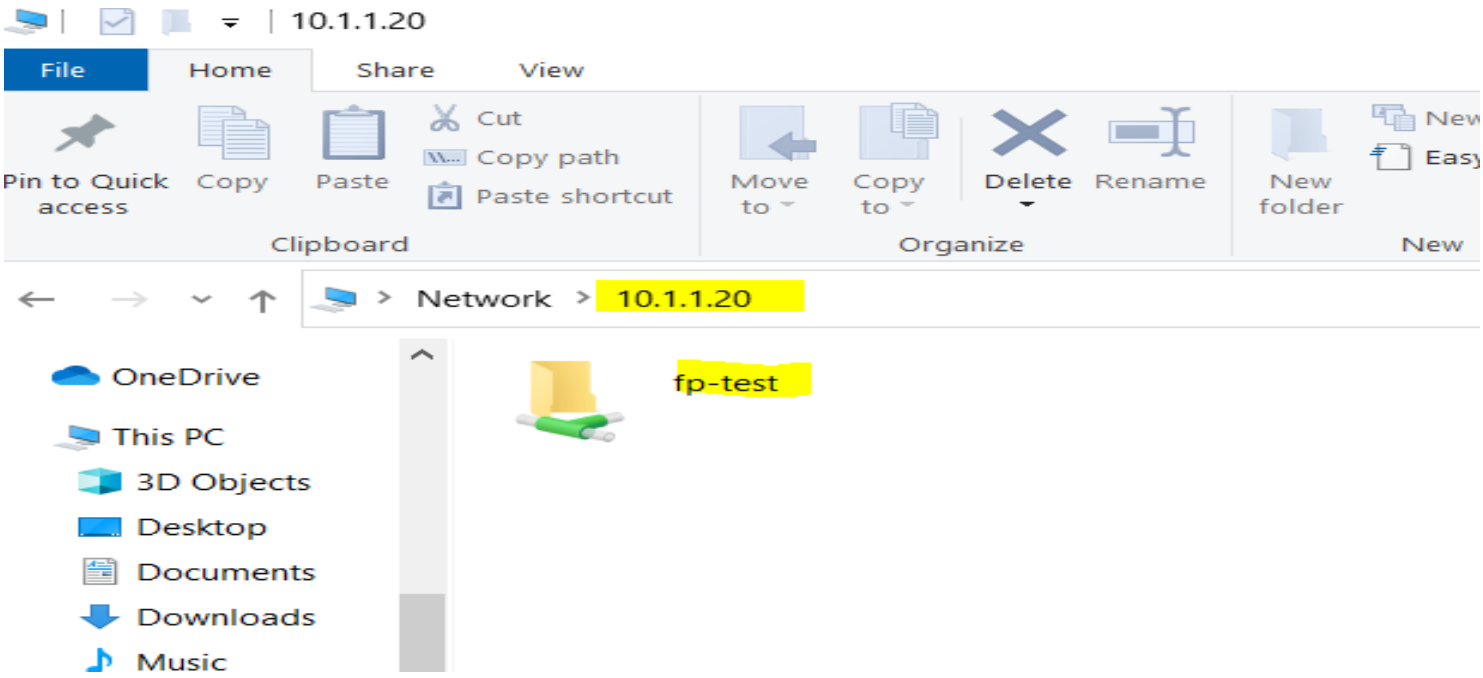
DXC-CLUSTER::> vserver fpolicy show
                                     Sequence
Vserver         Policy Name            Number  Status   Engine
--------------- -------------------- --------  -------- ---------
svm-senthil     ransom_block                1  on       native
```

Enabling Fpolicy in the vserver

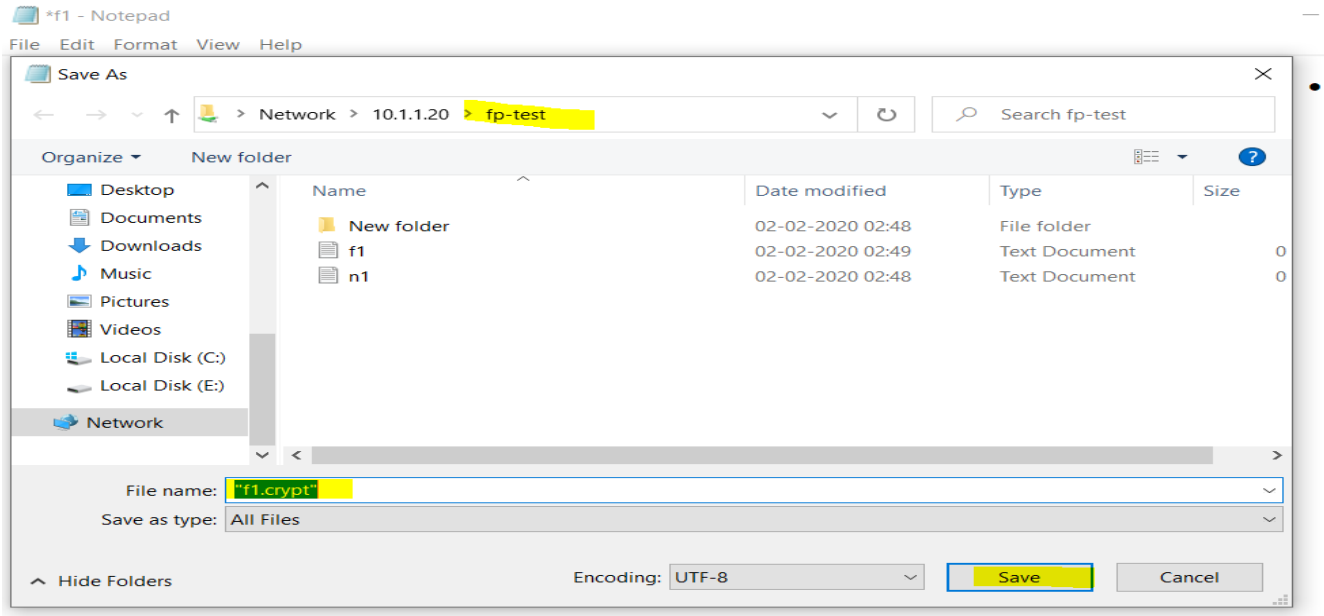
```
DXC-CLUSTER::> vserver fpolicy policy scope create -vserver svm-senthil -policy-
name ransom_block -file-extensions-to-include micro,encrypted,locked,crypto,cryp
t,crinf,r5a,XRNT,XTBL,R16M01D05,pzdc,good,LOL!,OMG!,RDM,RRK,encryptedRSA,crjoker
,EnCiPhErEd,LeChiffre,_crypt,Locky,SUPERCRYPT,CTBL,CTB2,WNCRY,ad4c,HD -volumes-t
o-include *

DXC-CLUSTER::> vserver fpolicy policy scope show
Vserver            Policy                Extensions            Extensions
Name               Name                  Included              Excluded
------------------ --------------------- --------------------- --------------------
svm-senthil        ransom_block          micro, encrypted,     -
                                         locked, crypto,
                                         crypt, crinf, r5a,
                                         XRNT, XTBL,
                                         R16M01D05, pzdc,
                                         good, LOL!, OMG!,
                                         RDM, RRK,
                                         encryptedRSA,
                                         crjoker, EnCiPhErEd,
                                         LeChiffre, _crypt,
                                         Locky, SUPERCRYPT,
                                         CTBL, CTB2, WNCRY,
                                         ad4c, HD
```

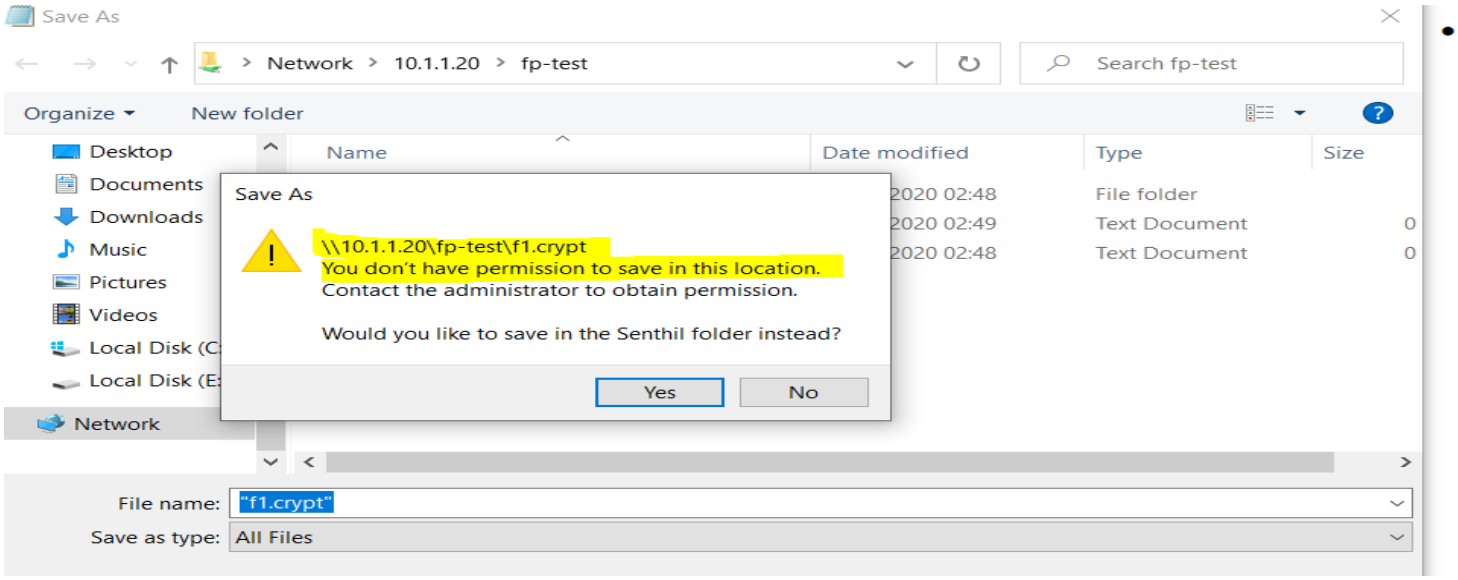
Creation of Fpolicy with the scopes



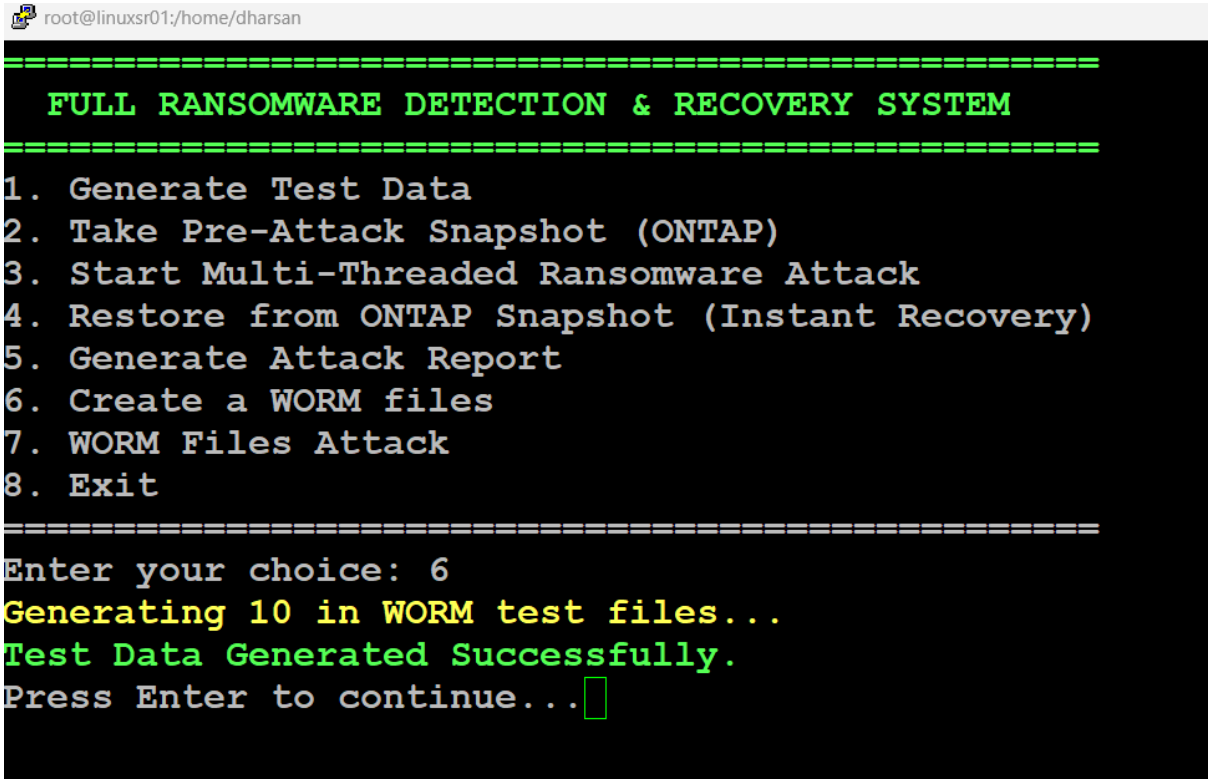
CIFS Share File



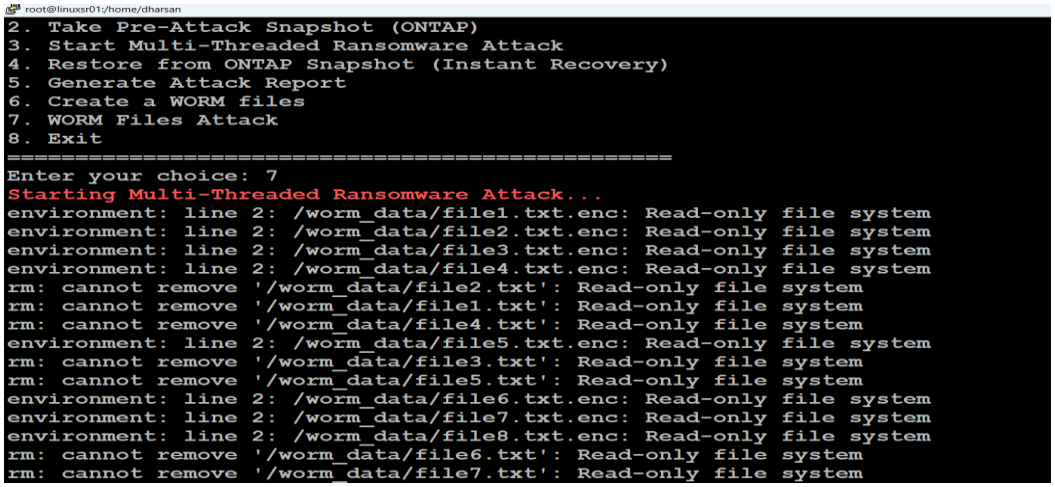
Creating Test files in the Shares file



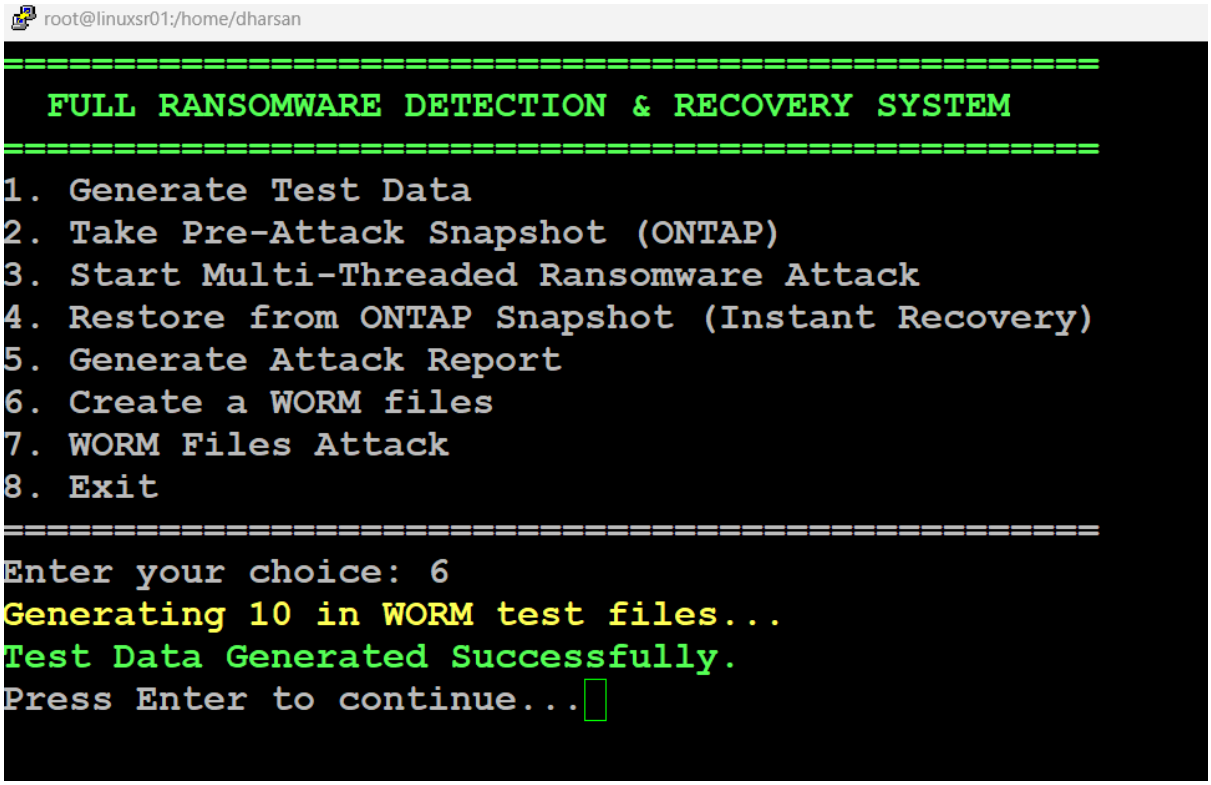
Fpolicy detects ransomware behavior and not allowing the user to change the extension of the file



Generating 10 test files in the snaplock using the bash script



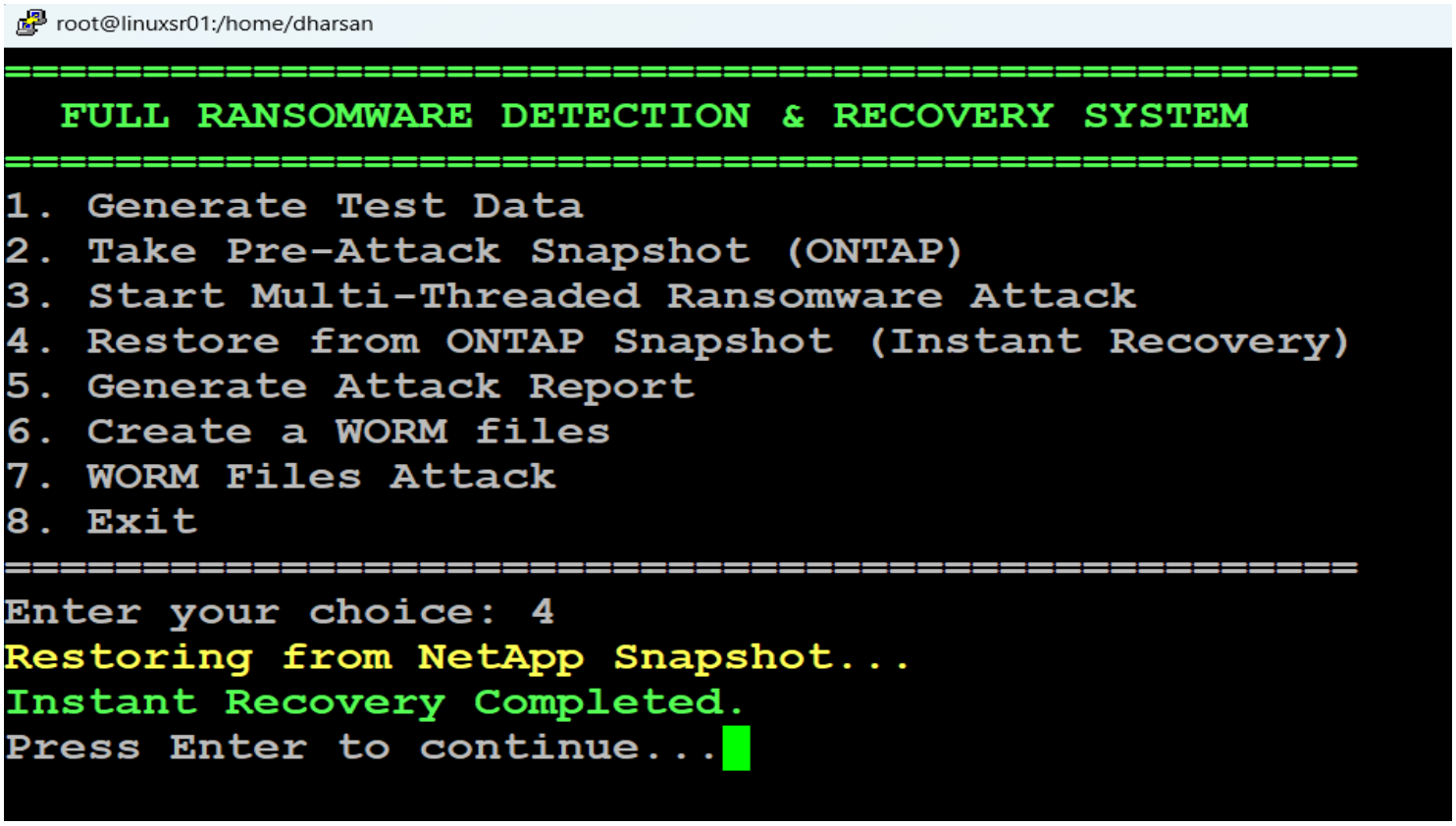
Protection of Files from Ransomware Attack in the snaplock or the WORM



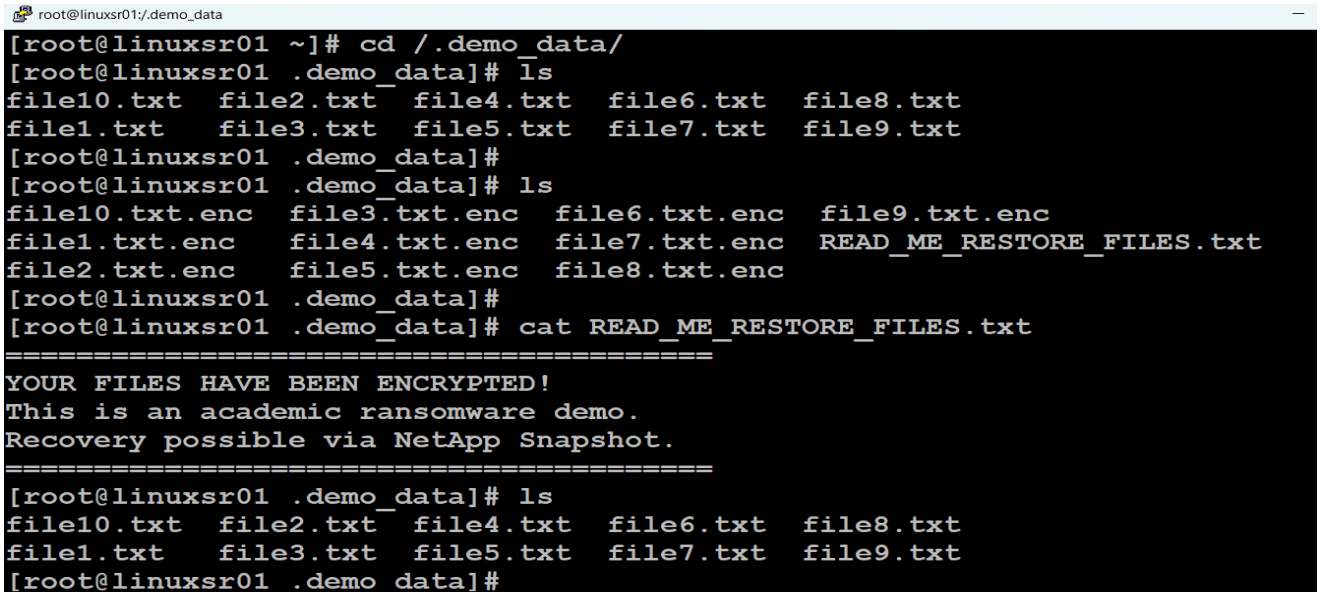
Generating 10 test files in the snaplock using the bash script

```
cluster01::>
cluster01::> volume snapshot show volnfs
                                                                  ---Blocks---
Vserver  Volume   Snapshot                                  Size Total% Used%
-------- -------- ------------------------------------ -------- ------ -----
svm01    volnfs
                  hourly.2026-02-15_1805                  152KB     0%   30%
                  pre_attack_snapshot                     164KB     0%   32%
2 entries were displayed.
```

Successfull creation of the pre attack snapshot



Recovery of the attacked files using the snapshot through the bash script



Successful Recovery of the attacked files using the snapshot through the bash script